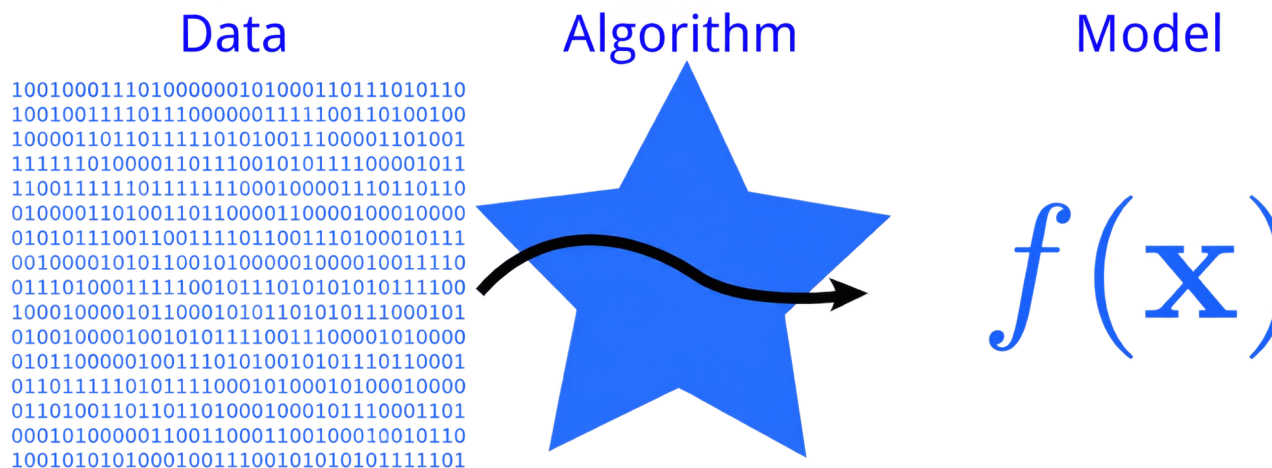


Machine Learning



Lecture 1: Introduction & regression

Iasonas Kokkinos

Imperial College London

i.kokkinos@ic.ac.uk

Course Requirements and Grading

Coursework (20%)

- Hands-on development of algorithms & experimentation
- Announced in week 3; due in week 7

Tutorials (0%)

- Weekly lab sessions
- Hands-on development of algorithms covered in class
- Critical for understanding the course

Final exam (80%)

- Theory questions (judgement-oriented)
- Simulate running algorithms by hand

Background

Prerequisites

Linear algebra, probability, calculus, and programming.
Refresh this background during the first week.

Rank, norm, determinant, likelihood, density, log-likelihood, gradient, ..

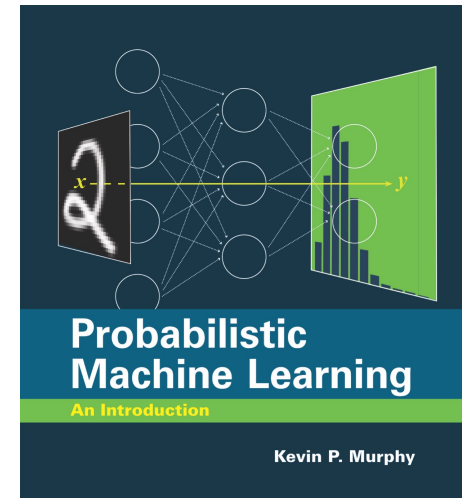
Kevin P. Murphy

Probabilistic Machine Learning: An Introduction
MIT Press, 2022

Refresher: “foundations” part

[Book website](#)

[Download the free PDF](#)



Office hours

Office 450, Monday, 5:00-6:00

Please send an email at least two hours ahead.

Strongly encouraged for:

- Questions regarding the course
- Clarifications
- Feedback

Feedback

Required to improve the course for this and next term - please do not wait until the end.

Please do it

- during/after the lecture
- during office hours
- online: <https://forms.cloud.microsoft/e/De8UK4392r>
- at the end

Course schedule

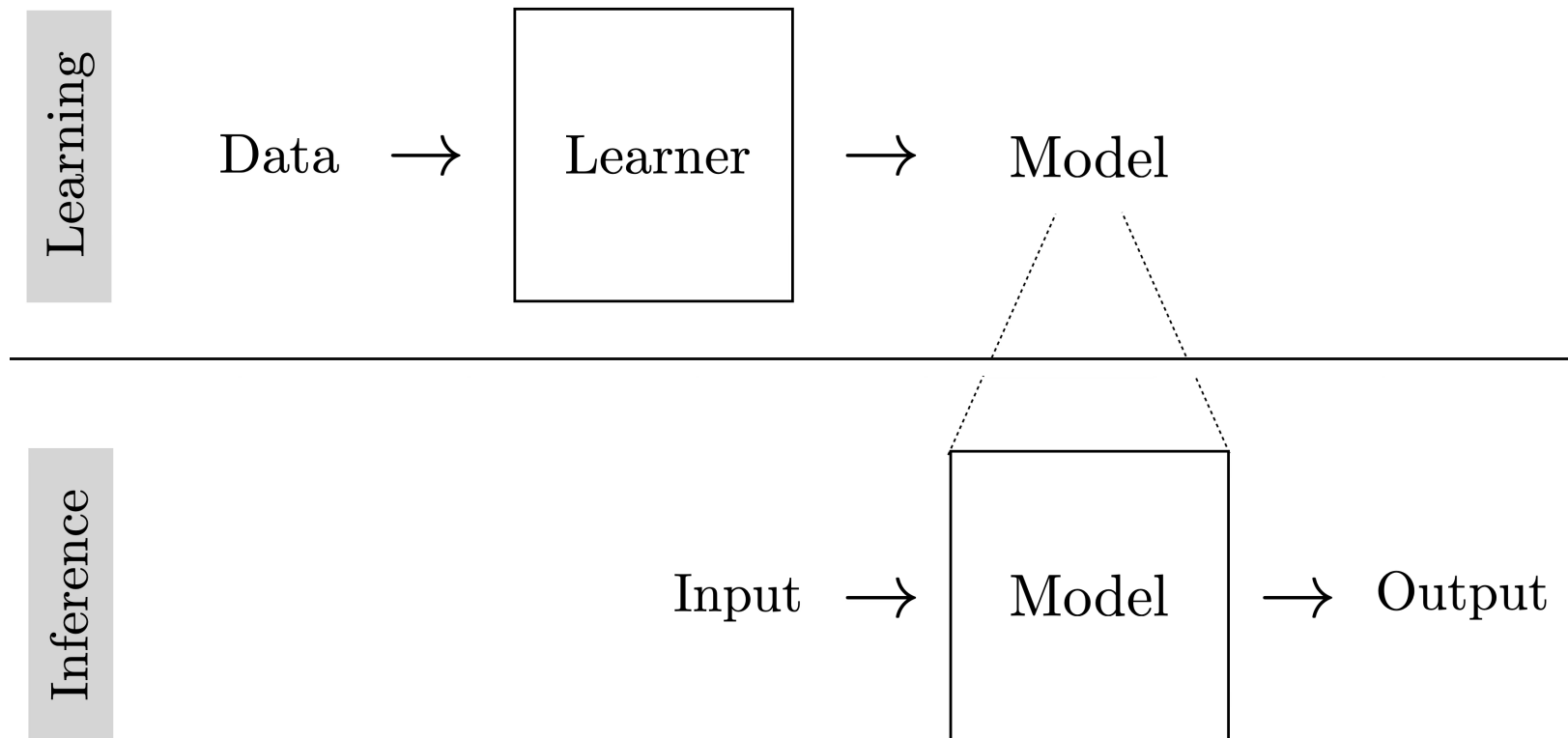
- 1st lecture: **Intro** & Regression
- 2nd lecture: Classification
- 3rd lecture: Neural Networks
- 4th lecture: Convolutional Neural Networks
- 5th lecture: Transformers
- 6th lecture: Unsupervised learning
- 7th lecture: Generative modelling
- 8th lecture: Revision & seminar

Lecture k = week $k+1$

Machine Learning

Machine learning is a field of **computer science** that gives computer systems the ability to **learn** (i.e., improve performance on a specific task) with **data**, without being explicitly programmed.

'ML' coined by Arthur Samuel, 1959.



Learning settings: the training signal

- Supervised** → An input and a supplied target.
- Unsupervised** → Observations without target labels.
- Self-supervised** → A target made from the observation itself.
- Partial supervision** → A few labels, or coarse or noisy labels.
- Reinforcement learning** → Feedback on actions and consequences.

Supervised learning: supplied targets

Music genre prediction

Training tracks come with measured sound features and supplied genre labels.

Track A	→	Dance
Track B	→	Acoustic
Track C	→	Classical



Learn to label a new track from its sound.

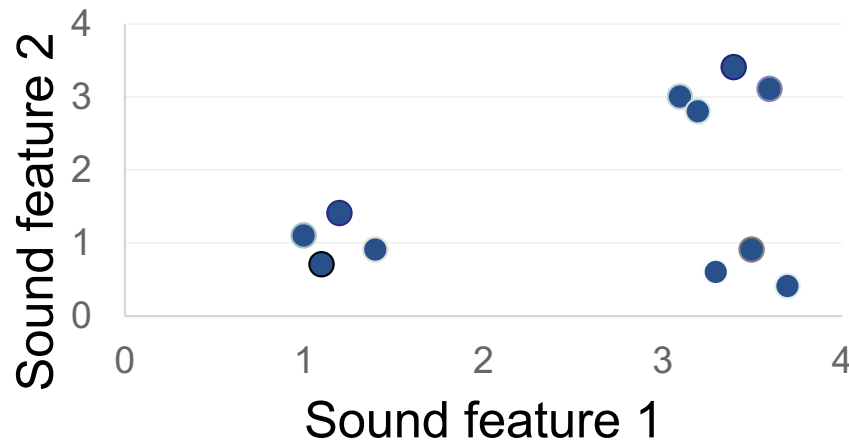
Other uses: spam filtering and recognising objects in photos.

Unsupervised learning: discover groups

Same music, no genre labels

Group tracks that sound similar.

The groups come from patterns in the observations.



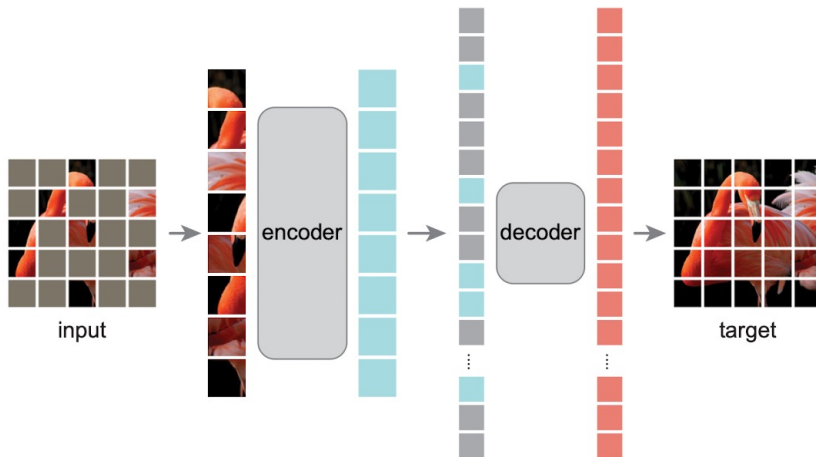
Each point is a track. The axes are measured sound features.

Applications: group music catalogues or organise photo collections.

Self-supervision: targets from data transforms

Hide image patches

Generate hidden patches from visible ones.



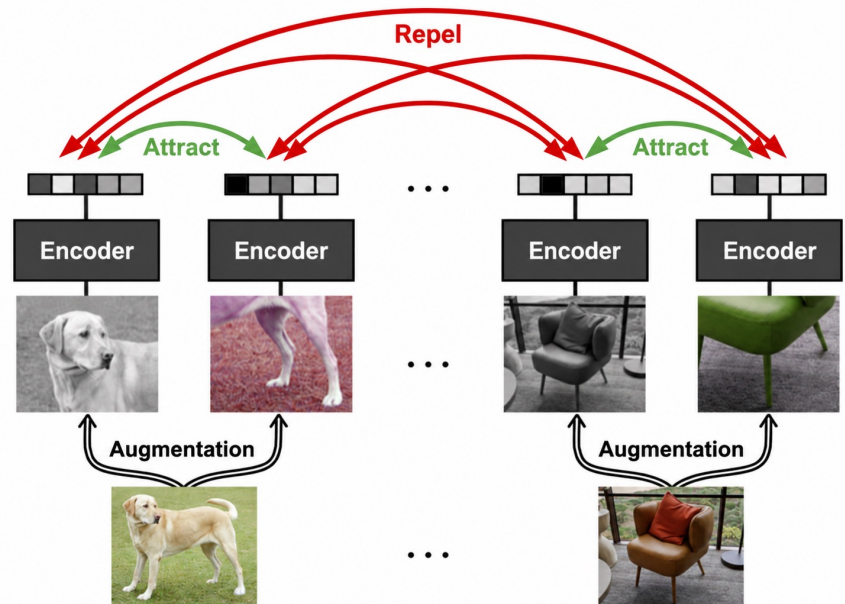
Hide words

“The detective understood it was .”

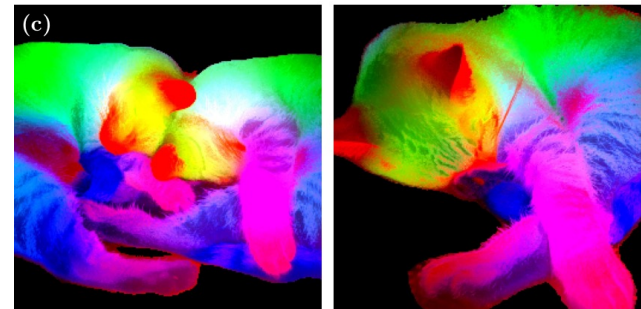
GPT: Generative Pre-trained Transformer

Match transformed views

Pull views of one image together.
Push different images apart.



DINOv3 (2025): image features



Partial supervision

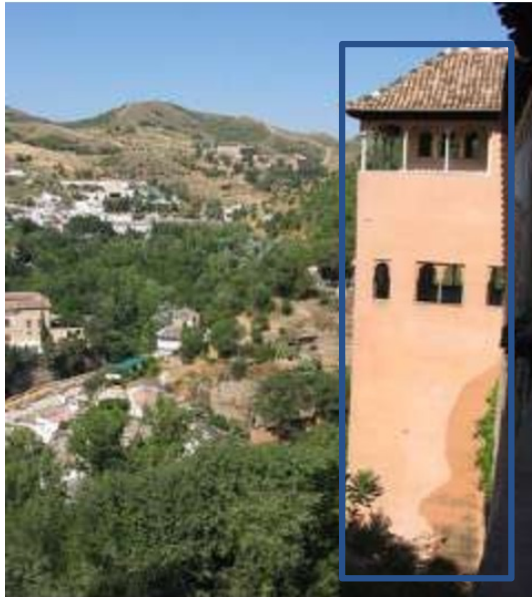
Same image, different ground-truth detail

Class label



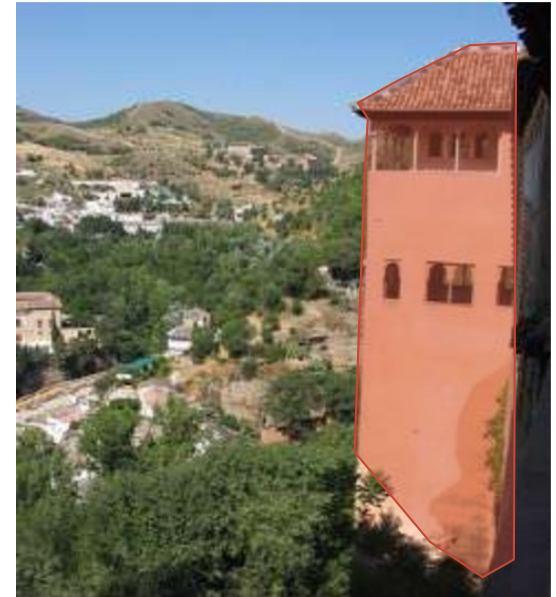
“Building present”
No location supplied.

Bounding box



Location and extent.
No pixel labels supplied.

Segmentation mask



Labels individual pixels.
Most detailed target.

Weak supervision: coarse labels leave target detail to infer.

Semi-supervision: combine labelled and unlabelled examples.

Reinforcement learning: actions and rewards

AlphaGo: learning through self-play



Observe the board.
Choose the next move.
Receive a reward for the outcome.

Learn a policy: a rule for choosing an action in each state.

An action can matter long before a reward arrives.
The objective concerns consequences over the whole game.

The focus of this course

Supervised problems

Classification and regression: predict a target from an input.

Unsupervised problems

Clustering, representation learning and generative modelling:
learn structure or a distribution from observations.

We study small models carefully.

The same ideas support much larger applications.

Course schedule

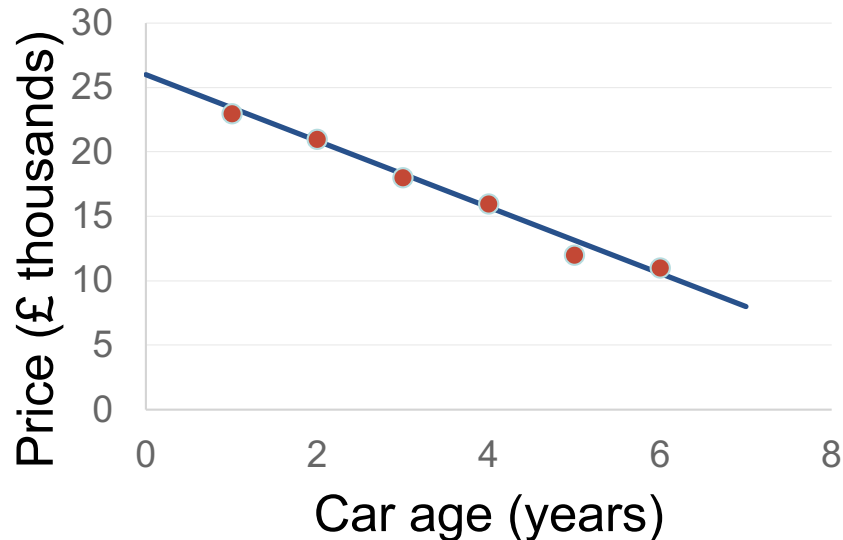
- 1st lecture: Intro & Regression
- 2nd lecture: Classification
- 3rd lecture: Neural Networks
- 4th lecture: Convolutional Neural Networks
- 5th lecture: Transformers
- 6th lecture: Unsupervised learning
- 7th lecture: Generative modelling
- 8th lecture: Revision & seminar

Lectures 1–2: supervised problems. Lectures 3–5: deep learning.
Buildup to unsupervised learning in 6 and generative models in 7.

Regression: predict a number

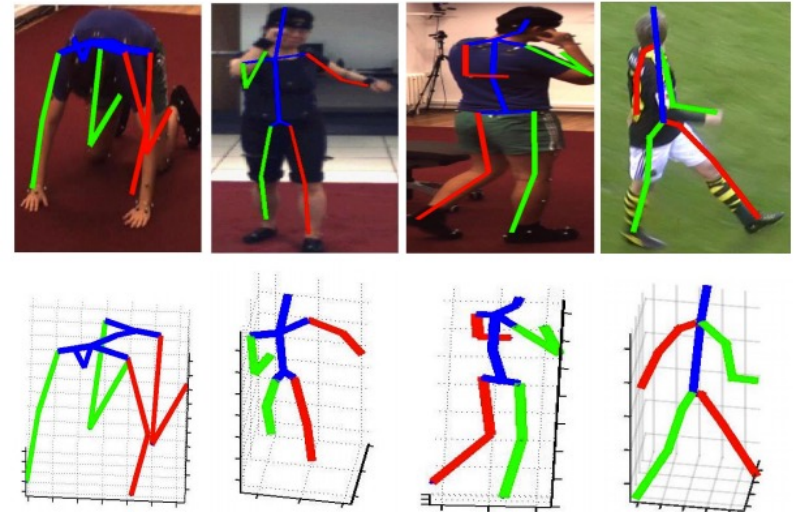
Minimal model

Fit a line to predict car price from age.



Real application

Predict the coordinates of body joints in an image.

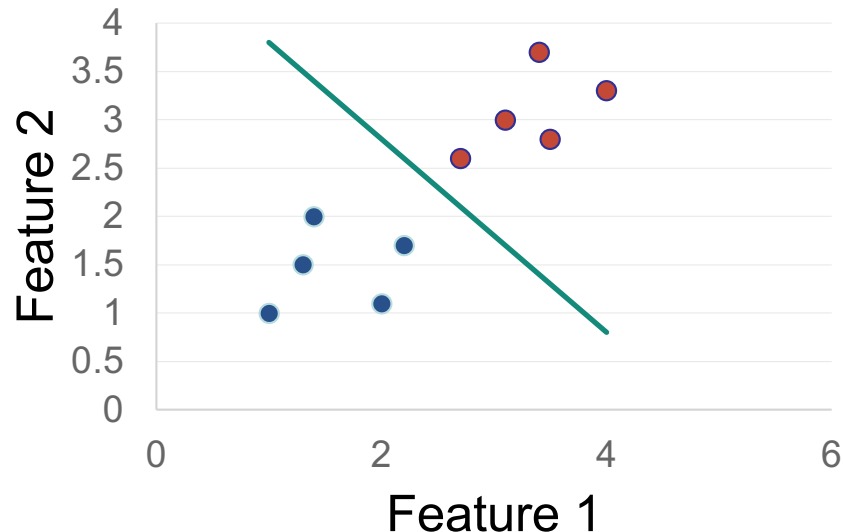


The output is numerical. A model can predict many numbers at once.

Classification: choose a category

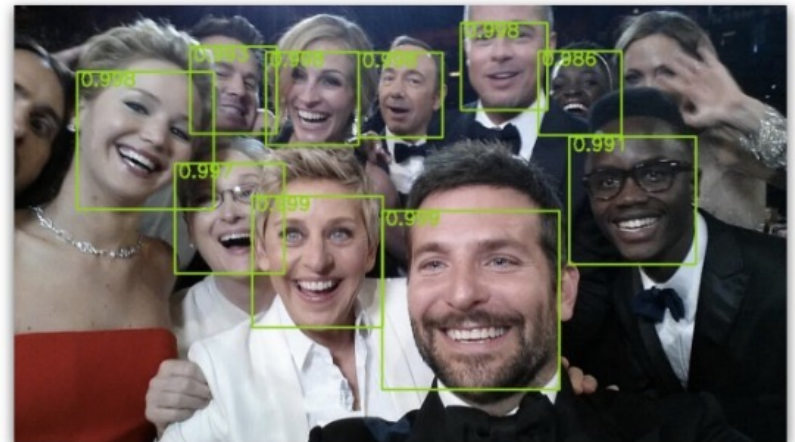
Minimal model

Separate two categories with a decision boundary.



Real application

Find faces at many positions and scales in a photo.

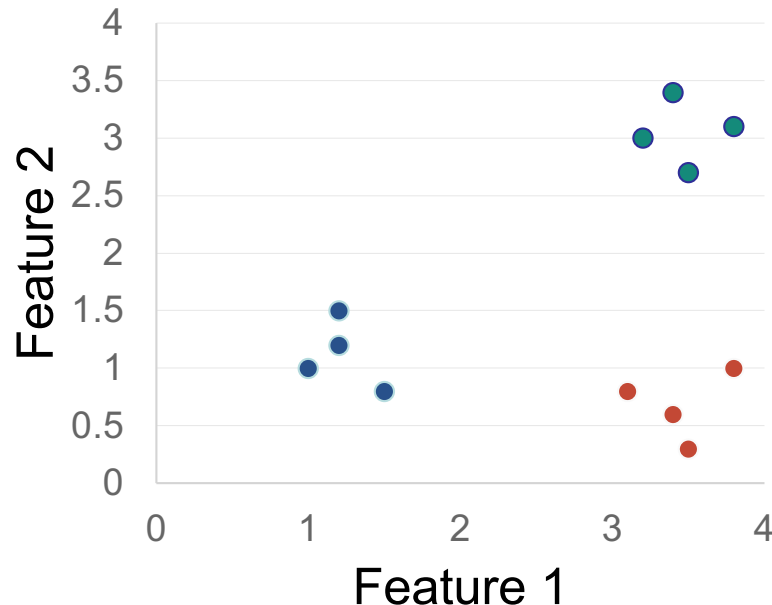


The output is a category. The input can be a small vector or an image.

Unsupervised learning: “invent labels”

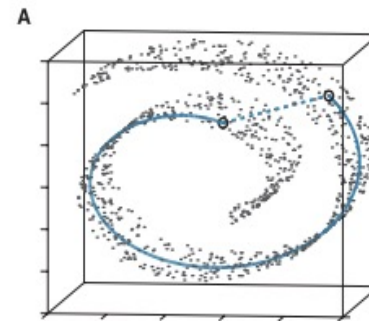
Clustering (discrete)

Choose group centres and assign nearby observations.

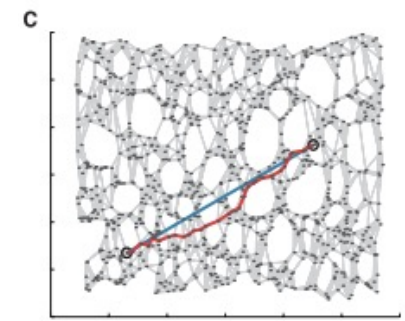


Dimensionality reduction (continuous)

Find a compact set of coordinates that captures data variation.



3D manifold



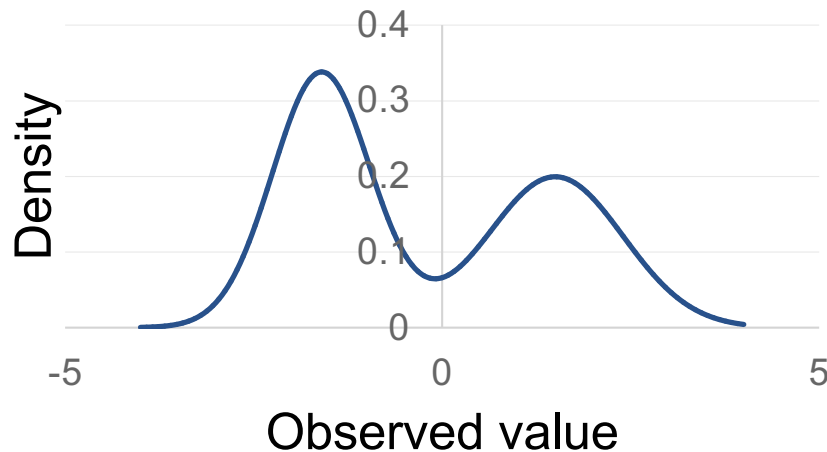
2D coordinates

Clustering assigns discrete groups. Dimensionality reduction assigns continuous coordinates, using the observations alone.

Generative modelling: learn a distribution

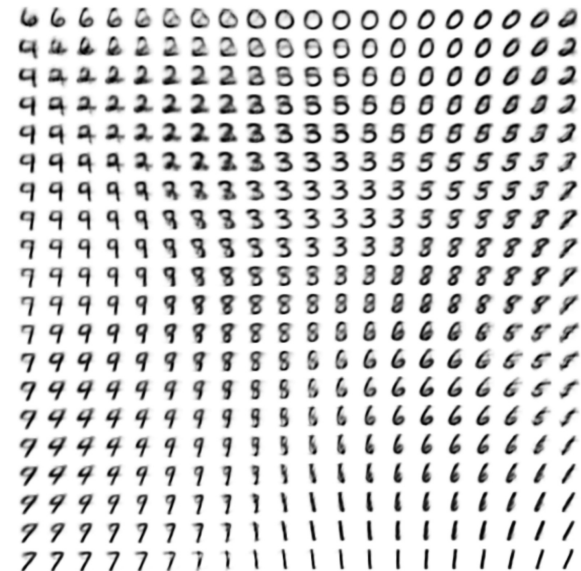
Minimal model

A mixture of Gaussians models several groups with bell-shaped densities.



A small digit model

A variational autoencoder (VAE) learns hidden coordinates and generates digits.



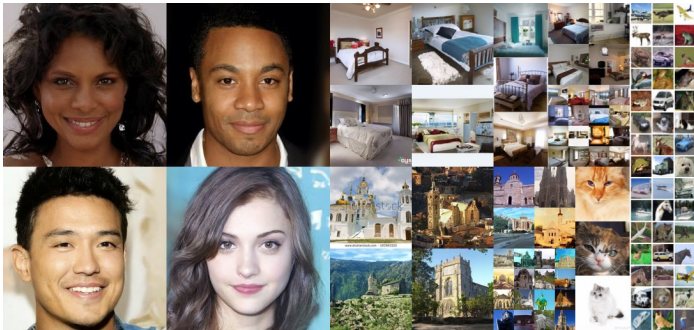
Learning describes which observations are plausible.

Generation creates new observations from the learned model.

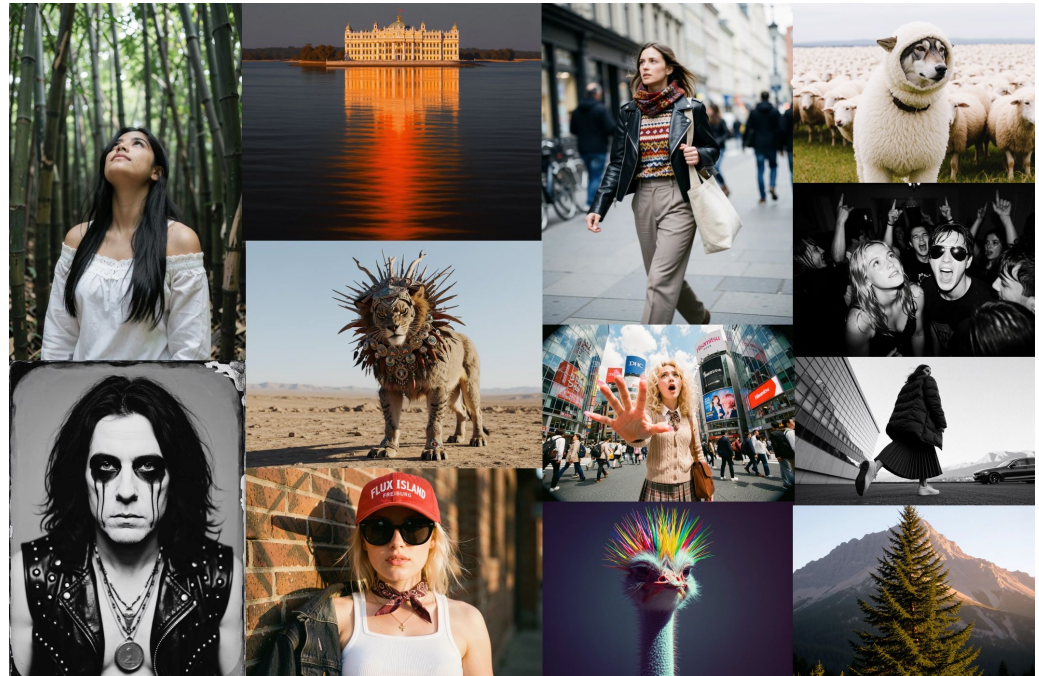
Generative modelling: from noise to images

Learn a transformation from noise to plausible images.

DDPM (2020)



FLUX.2 [klein] (2026)



Both transform noise into plausible image samples from a learned data distribution.

Course schedule

- 1st lecture: Intro & **Regression**
- 2nd lecture: Classification
- 3rd lecture: Neural Networks
- 4th lecture: Convolutional Neural Networks
- 5th lecture: Transformers
- 6th lecture: Unsupervised learning
- 7th lecture: Generative modelling
- 8th lecture: Revision & seminar

Lecture 1: regression

Regression

Local averaging

Linear regression: least squares and nonlinear features

Ridge regression

Generalisation and evaluation

Empirical risk, bias–variance, validation and evaluation

Probability and ridge regression

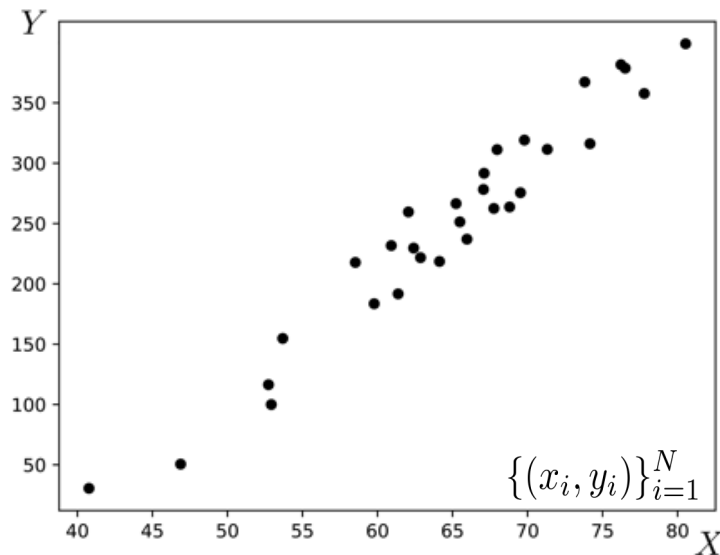
Probability basics and Maximum Likelihood (ML) estimation

Ridge regression as Maximum a Posteriori (MAP) estimation

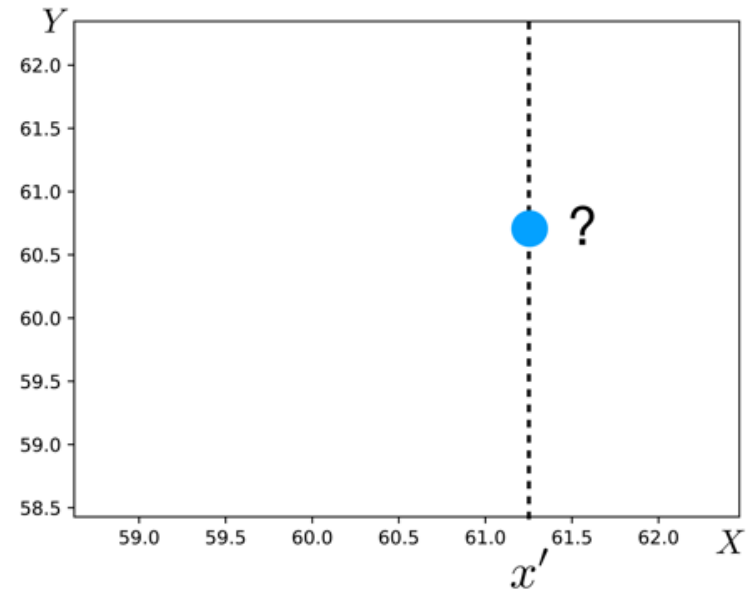
Regression: a new input, an unknown target

We observe input–target pairs. What should we predict for a new input?

Training data



Test query

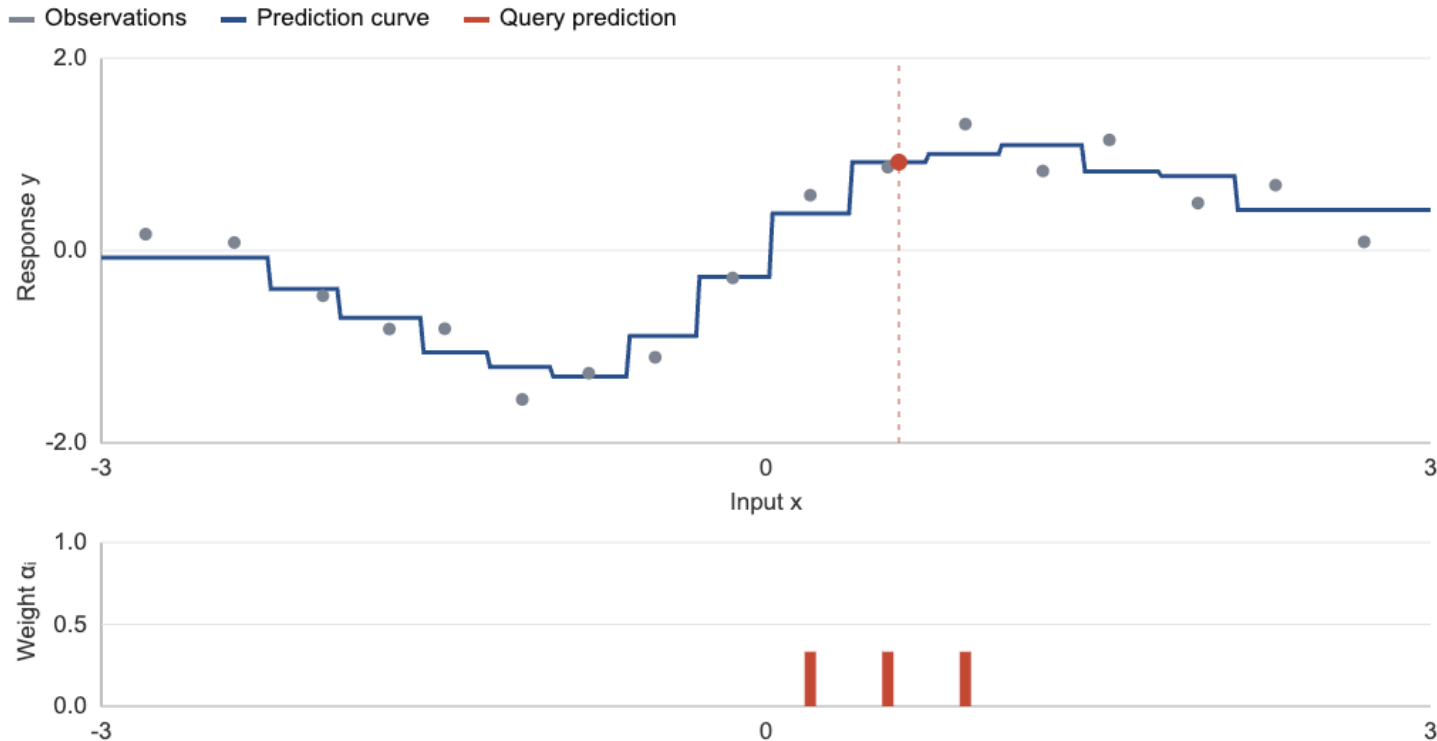


X and Y label input and target. x' is a new input.

Its target is unknown. The blue marker's height is only a placeholder.

Non-parametric regression: local averaging

$k = 3$: average the targets of the three nearest stored inputs.



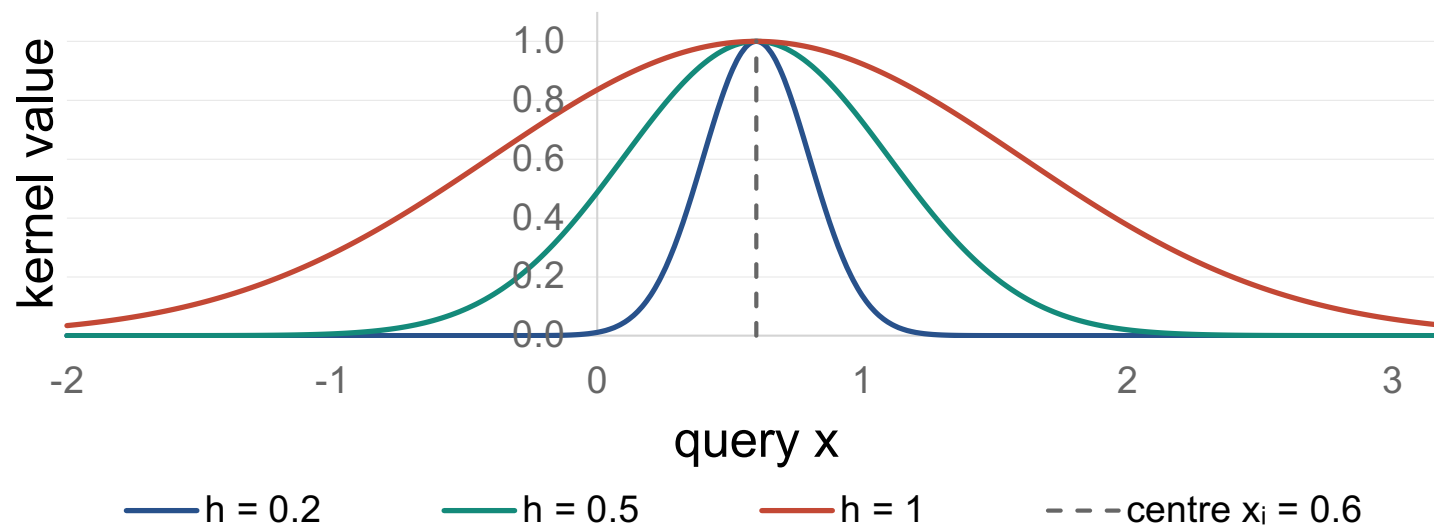
Each neighbour has weight $1/3$. Grey: observations. Blue: prediction curve. Red: prediction at the query.

[Open the interactive demo in a browser](#)

A kernel assigns weights by distance

$$K_h(x - x_i) = \exp\left[-\frac{(x_i - x)^2}{2h^2}\right], \quad h > 0$$

x is the query. x_i is stored input i . $h > 0$ is the kernel width.



$$K_h(\pm h) = e^{-1/2} \approx 0.61$$

Each kernel is centred at its stored input x_i (here $x_i = 0.6$).
Larger h gives distant inputs more influence. A Gaussian has no hard cutoff.

Kernel regression: a weighted average

$$\hat{y}(x) = \sum_{i=1}^N \alpha_i(x) y_i, \quad \alpha_i(x) = \frac{K_h(x - x_i)}{\sum_{j=1}^N K_h(x - x_j)}$$

h = 0.12

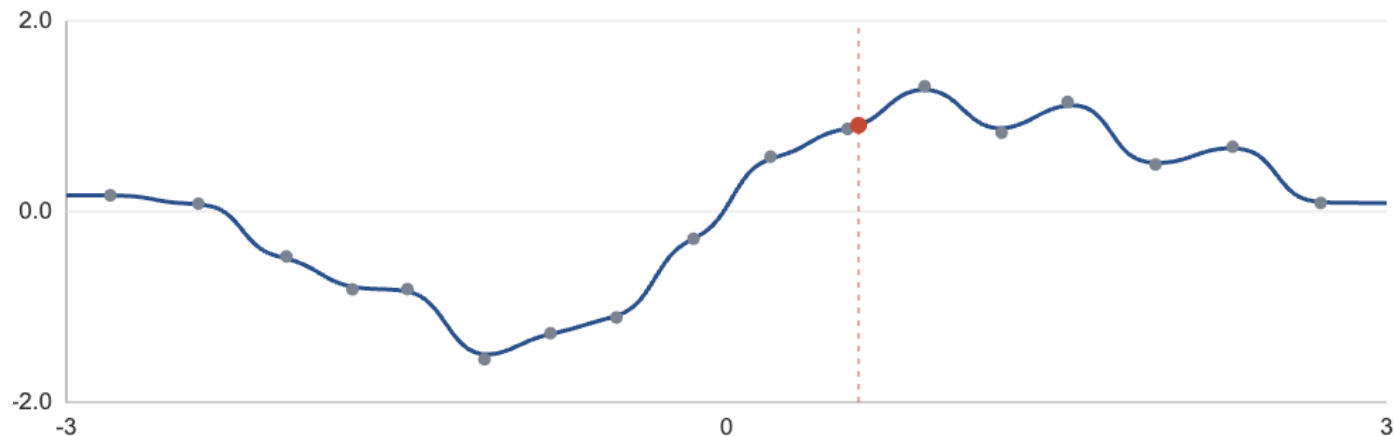
$\alpha_i(x)$: normalised target weights. N = 17.

Grey: observations

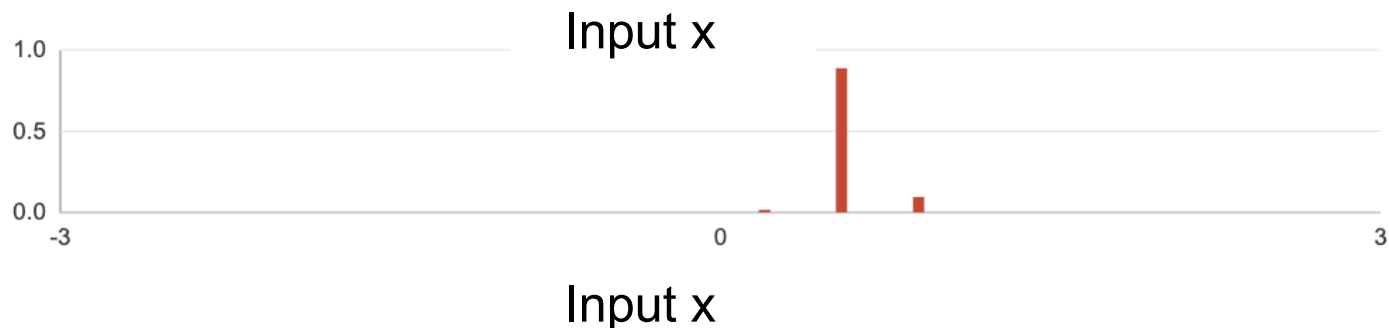
Blue: prediction

Red: query

Response
y



Weights
 $\alpha_i(x)$



Kernel regression: a weighted average

$$\hat{y}(x) = \sum_{i=1}^N \alpha_i(x) y_i, \quad \alpha_i(x) = \frac{K_h(x - x_i)}{\sum_{j=1}^N K_h(x - x_j)}$$

h = 0.5

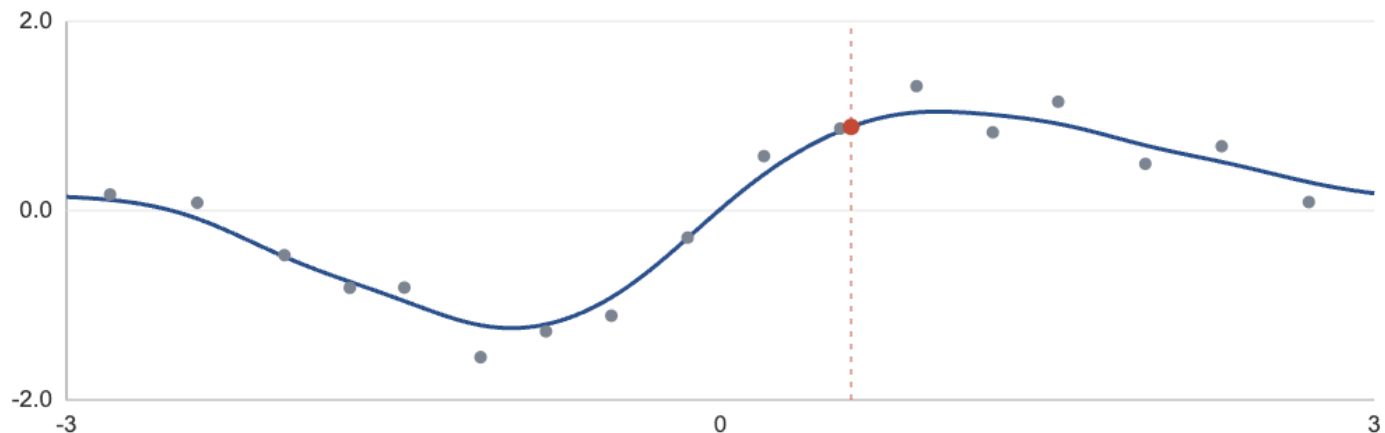
$\alpha_i(x)$: normalised target weights. N = 17.

Grey: observations

Blue: prediction

Red: query

Response
y



Weights
 $\alpha_i(x)$



Input x

Challenges of non-parametric regression

Memory

Storing the training pairs grows linearly with sample count for a fixed input dimension.

Prediction

Access the stored examples. A straightforward full scan grows linearly with sample count. An index may be faster.

Locality

The curse of dimensionality: neighbourhoods become sparse. Below: 30 observations, 10 bins per coordinate.

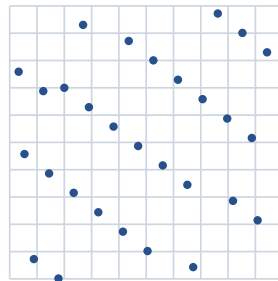
1 dimension



10 cells

3 per cell on average

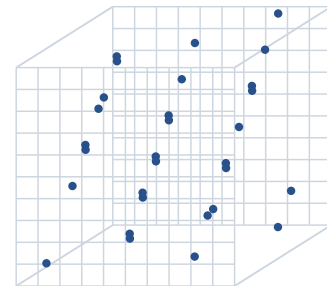
2 dimensions



100 cells

0.3 per cell on average

3 dimensions



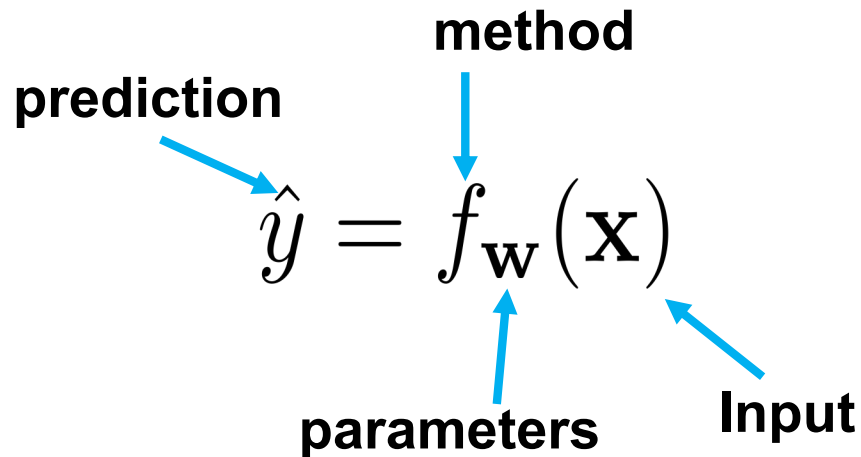
1,000 cells

0.03 per cell on average

Inputs, parameters and predictions

A parametric model stores a fixed-size parameter vector.

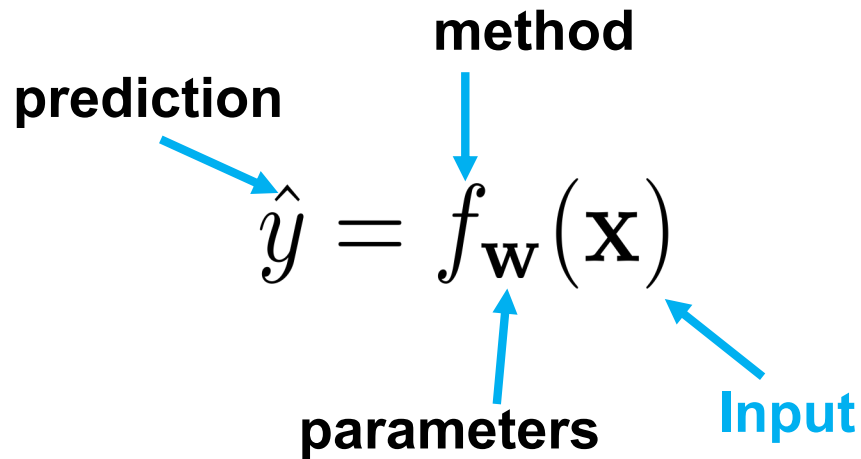
Prediction evaluates the fitted function without retrieving training examples.



f : prediction function. w : its parameters. x : input. $\hat{y}$: predicted output.

A hat marks an estimate. Bold letters denote vectors.

Representing the input



Scalar input $x \in \mathbb{R}$

Vector input $\mathbf{x} \in \mathbb{R}^D$

$\mathbb{R}$ denotes real numbers. D is the number of input coordinates.

Inputs can also be discrete: strings, trees or graphs.

Parameterising the predictor

- Input-output mapping

The diagram shows the equation $\hat{y} = f_{\mathbf{w}}(\mathbf{x}) = f(\mathbf{x}; \mathbf{w})$. Four blue arrows point to different parts of the equation: one from the word 'prediction' to $\hat{y}$, one from the word 'method' to f , one from the word 'parameters' to $\mathbf{w}$, and one from the word 'Input' to $\mathbf{x}$.

$$\hat{y} = f_{\mathbf{w}}(\mathbf{x}) = f(\mathbf{x}; \mathbf{w})$$

prediction method parameters Input

$$w \in \mathbb{R}$$

$$\mathbf{w} \in \mathbb{R}^M$$

M: number of coefficients.

The semicolon separates inputs from parameters.

Linear regression: a linear predictor

$$\hat{y} = f_{\mathbf{w}}(\mathbf{x}) = f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \mathbf{x}$$

Inner product:

$$\mathbf{w}^T \mathbf{x} = \langle \mathbf{w}, \mathbf{x} \rangle = \sum_{j=1}^D w_j x_j$$

$$\mathbf{x} \in \mathbb{R}^D, \mathbf{w} \in \mathbb{R}^D$$

j: input coordinate index. D: number of coordinates. $\langle \mathbf{w}, \mathbf{x} \rangle$: inner product.
 $\sum$ sums over coordinates. T denotes transpose.

Fitting a model to data

We observe N input–target pairs.

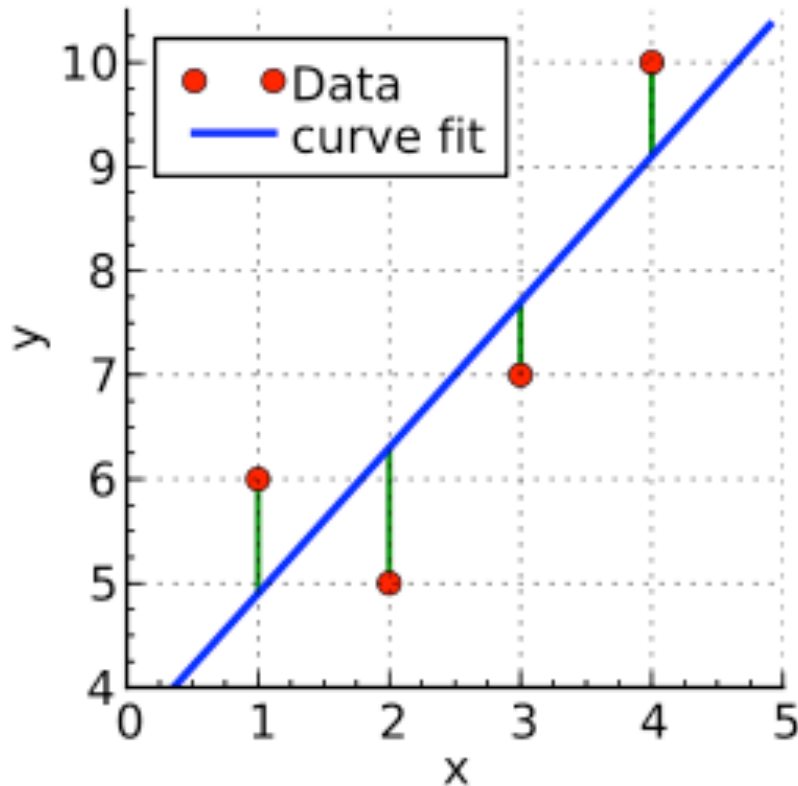
$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$$

We fit the parameter vector $\mathbf{w}$ to these data.

$$\hat{y} = f_{\mathbf{w}}(\mathbf{x})$$

Choose $\mathbf{w}$ so the predictions are close to the observed targets.

Linear regression in 1D



$$\hat{y}_i = w_0 + w_1 x_i$$

w_0 is the intercept. w_1 is the slope.
 i indexes a training pair.
 x_i is its scalar input and y_i its target

$$\epsilon_i = y_i - \hat{y}_i = y_i - (w_0 + w_1 x_i)$$

ϵ_i is residual i : the signed vertical difference between target and prediction.

Least squares criterion

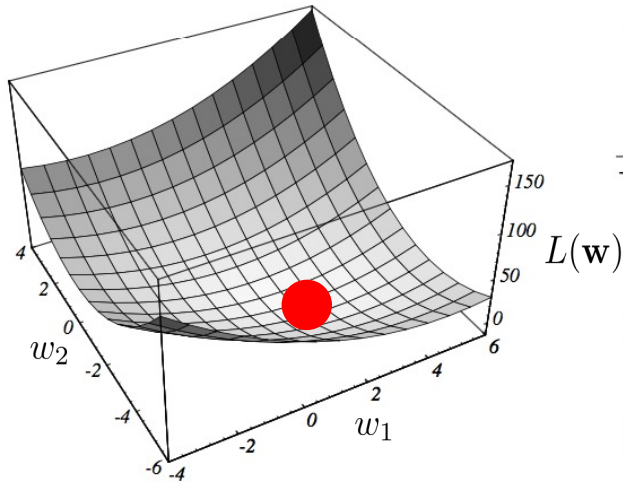
Residual sum of squares (RSS) measures training fit.

$$L(\mathbf{w}) = \sum_{i=1}^N \epsilon_i^2 = \sum_{i=1}^N (y_i - w_0 - w_1 x_i)^2$$

$$\hat{\mathbf{w}} \in \arg \min_{\mathbf{w}} L(\mathbf{w})$$

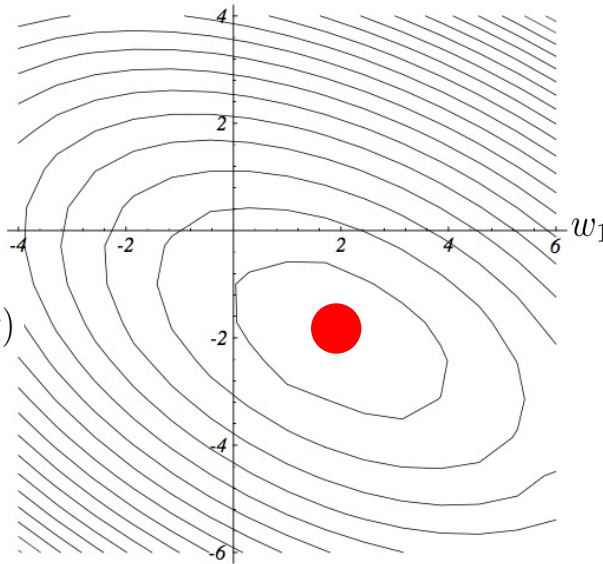
Least squares chooses the coefficient vector that minimises $L(\mathbf{w})$.
The hat marks the fitted parameters.

Gradient-based optimization



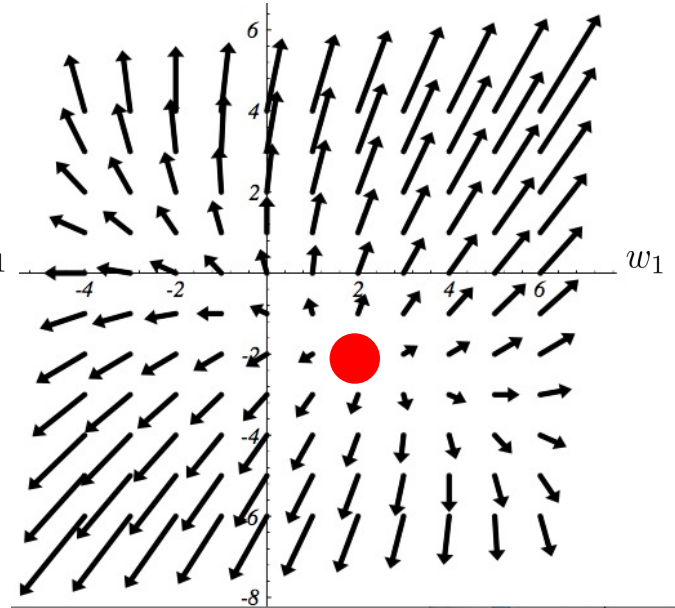
$$L(\mathbf{w})$$

2D function graph



$$L(\mathbf{w}) = c$$

isocontours



$$\nabla L(\mathbf{w}) = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \end{bmatrix}$$

gradient field

● at minimum: $\nabla L(\mathbf{w}) = \mathbf{0}$

L is the loss. ∇L is its gradient. c is a contour value.

Differentiating through the residual

$$L(\mathbf{w}) = \sum_{i=1}^N \epsilon_i^2, \quad \epsilon_i = y_i - w_0 - w_1 x_i$$

L is RSS. ϵ_i is residual i . ∂ differentiates one coefficient.

$$\frac{\partial \epsilon_i}{\partial w_0} = -1, \quad \frac{\partial \epsilon_i}{\partial w_1} = -x_i$$

$$\begin{aligned} \frac{\partial L}{\partial w_0} &= \sum_{i=1}^N 2\epsilon_i \frac{\partial \epsilon_i}{\partial w_0} = -2 \sum_{i=1}^N \epsilon_i \\ &= -2 \sum_{i=1}^N (y_i - w_0 - w_1 x_i) \end{aligned}$$

Fitting a line: stationary conditions

Set both partial derivatives to zero

$$\frac{\partial L}{\partial w_0} = -2 \sum_{i=1}^N (y_i - w_0 - w_1 x_i) = 0$$

$$\frac{\partial L}{\partial w_1} = -2 \sum_{i=1}^N x_i (y_i - w_0 - w_1 x_i) = 0$$

Rearrange
$$Nw_0 + w_1 \sum_i x_i = \sum_i y_i$$

$$w_0 \sum_i x_i + w_1 \sum_i x_i^2 = \sum_i x_i y_i$$

Introduce the intercept pseudo-coordinate $x_{i0} = 1$ and let $x_{i1} = x_i$.
Write the same equations as a linear system.

$$\begin{bmatrix} \sum_i x_{i0}^2 & \sum_i x_{i0} x_{i1} \\ \sum_i x_{i0} x_{i1} & \sum_i x_{i1}^2 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} \sum_i x_{i0} y_i \\ \sum_i x_{i1} y_i \end{bmatrix}$$

Matrix form of the normal equations

$$\begin{bmatrix} \sum_i y_i x_{i0} \\ \sum_i y_i x_{i1} \end{bmatrix} = \begin{bmatrix} \sum_i x_{i0}^2 & \sum_i x_{i0} x_{i1} \\ \sum_i x_{i0} x_{i1} & \sum_i x_{i1}^2 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{y} = \mathbf{X}^T \mathbf{X} \mathbf{w}$$

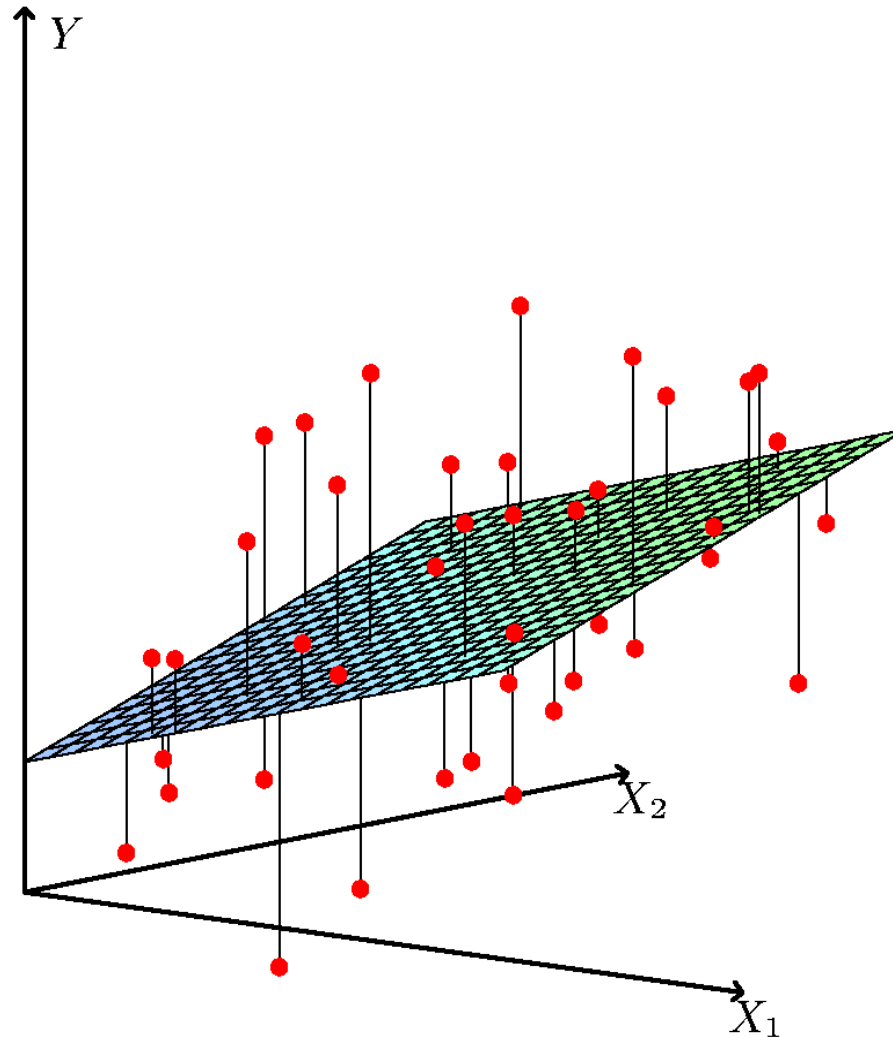
$$\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_N \end{bmatrix}$$

Normal Equation

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

$\mathbf{X}$ is the design matrix; $\mathbf{y}$ stacks targets. Full column rank means independent columns and makes $\mathbf{X}^T \mathbf{X}$ invertible.

Linear regression in 2D (or ND)



Linear regression: the matrix model

N examples, D input coordinates, and one intercept column.

$$\begin{array}{c} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \\ N \times 1 \end{array} = \begin{array}{c} \begin{bmatrix} 1 & x_{11} & \cdots & x_{1D} \\ 1 & x_{21} & \cdots & x_{2D} \\ \vdots & \vdots & & \vdots \\ 1 & x_{N1} & \cdots & x_{ND} \end{bmatrix} \\ N \times (D+1) \end{array} \begin{array}{c} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_D \end{bmatrix} \\ (D+1) \times 1 \end{array} + \begin{array}{c} \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_N \end{bmatrix} \\ N \times 1 \end{array}$$

$$\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\epsilon}$$

X is the design matrix. y stacks targets, w coefficients, ϵ residuals.
The dimension labels give rows $\times$ columns.

Squared error as a vector norm

$$L(\mathbf{w}) = \sum_{i=1}^N \epsilon_i^2$$

$$\boldsymbol{\epsilon}^T \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_1 & \epsilon_2 & \cdots & \epsilon_N \end{bmatrix} \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_N \end{bmatrix}$$

$$L(\mathbf{w}) = \|\boldsymbol{\epsilon}\|_2^2 = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$$

$\|\boldsymbol{\epsilon}\|_2$ is the Euclidean norm. Its square sums the squared residuals.

Least squares solution, in vector form

$$\begin{aligned} L(\mathbf{w}) &= \boldsymbol{\epsilon}^T \boldsymbol{\epsilon} \\ &= (\mathbf{y} - X\mathbf{w})^T (\mathbf{y} - X\mathbf{w}) \\ &= \mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T X\mathbf{w} + \mathbf{w}^T X^T X\mathbf{w} \end{aligned}$$

Condition for minimum: $\nabla L(\hat{\mathbf{w}}) = \mathbf{0}$

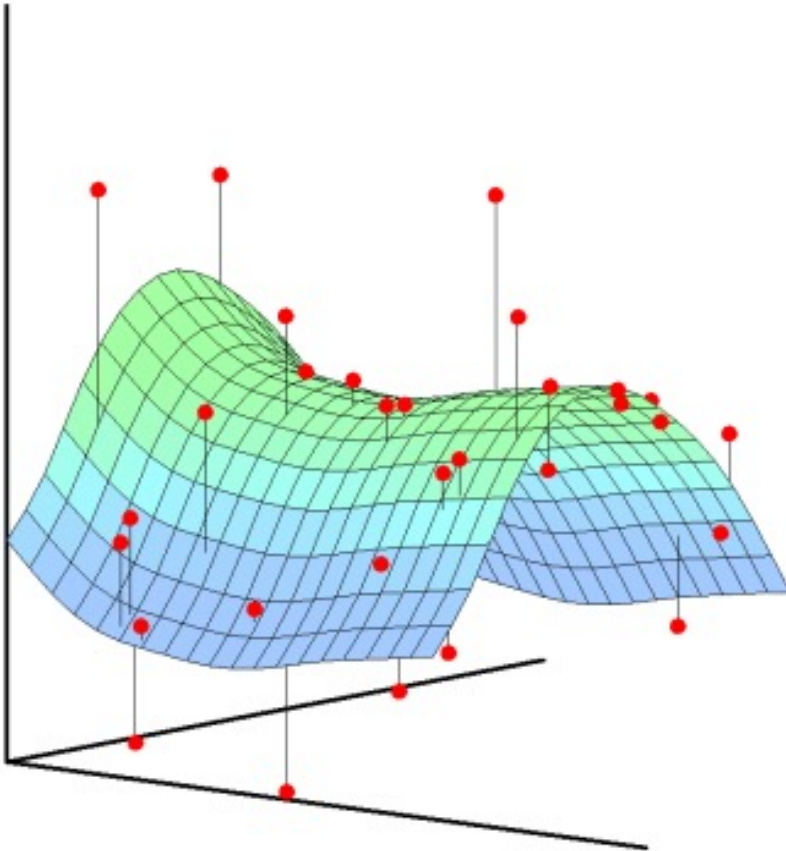
$$-2X^T \mathbf{y} + 2X^T X \hat{\mathbf{w}} = \mathbf{0}$$

$$\hat{\mathbf{w}} = (X^T X)^{-1} X^T \mathbf{y}$$

The hat marks the fitted coefficient vector.

With full column rank, this is the unique least-squares minimum.

Features



$$\mathbf{x} \mapsto \boldsymbol{\phi}(\mathbf{x}) = \begin{bmatrix} \phi_1(\mathbf{x}) \\ \vdots \\ \phi_M(\mathbf{x}) \end{bmatrix}$$

ϕ is the feature map. ϕ_j is feature j , for $j = 1, \dots, M$.
The first feature $\phi_1 = 1$ pairs with the intercept w_0 .

1D Example: 2nd degree polynomial fitting

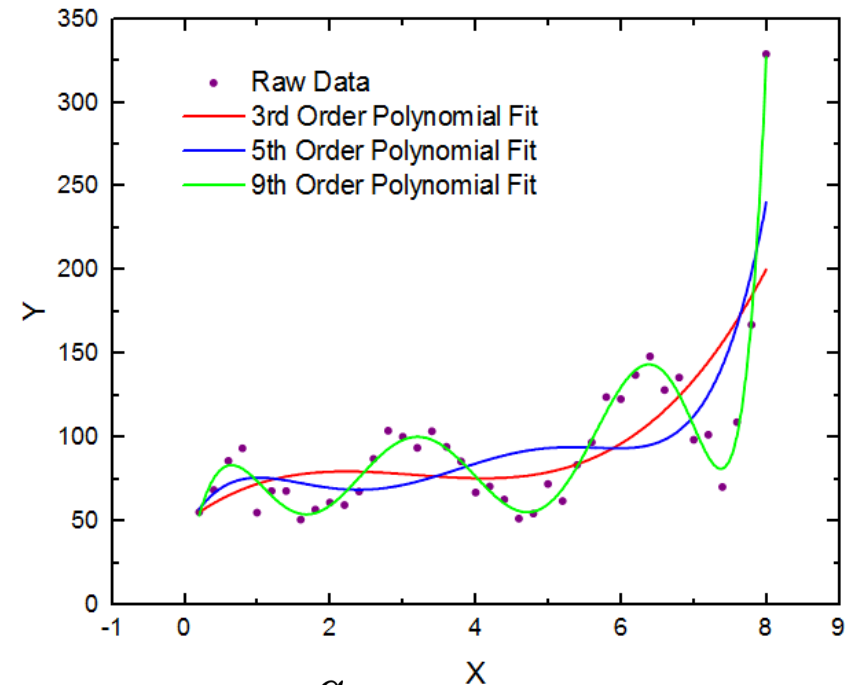
$$\phi(x) = \begin{bmatrix} \phi_1(x) \\ \phi_2(x) \\ \phi_3(x) \end{bmatrix} = \begin{bmatrix} 1 \\ x \\ x^2 \end{bmatrix}$$

$$\mathbf{w}^T \phi(x) = w_0 + w_1 x + w_2 x^2$$

Three features: $\phi_1(x) = 1$, $\phi_2(x) = x$, $\phi_3(x) = x^2$.
Their coefficients remain w_0 , w_1 and w_2 .

Polynomial features of degree q

$$\phi(x) = \begin{bmatrix} \phi_1(x) \\ \phi_2(x) \\ \vdots \\ \phi_{q+1}(x) \end{bmatrix} = \begin{bmatrix} 1 \\ x \\ \vdots \\ x^q \end{bmatrix}$$



$$\mathbf{w}^T \phi(x) = w_0 + w_1 x + \cdots + w_q x^q$$

q is the polynomial degree. M = q + 1 features, including $\phi_1 = 1$.

2D example: second-order polynomials

$$\mathbf{x} = (x_1, x_2)$$

$$\phi(\mathbf{x}) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_1^2 \\ x_2^2 \\ x_1 x_2 \end{bmatrix}$$

$$\mathbf{w}^T \phi(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_2^2 + w_5 x_1 x_2$$

A design matrix for nonlinear features

$$\hat{y}_i = f(\mathbf{x}_i; \mathbf{w}) = \mathbf{w}^T \phi(\mathbf{x}_i)$$

Linear in the coefficients $\mathbf{w}$, with potentially nonlinear input features.

$$\Phi = \begin{bmatrix} \phi_1(\mathbf{x}_1) & \cdots & \phi_M(\mathbf{x}_1) \\ \vdots & & \vdots \\ \phi_1(\mathbf{x}_N) & \cdots & \phi_M(\mathbf{x}_N) \end{bmatrix} \in \mathbb{R}^{N \times M}$$

$$\mathbf{y} = \Phi \mathbf{w} + \boldsymbol{\epsilon}, \quad \mathbf{y}, \boldsymbol{\epsilon} \in \mathbb{R}^N, \quad \mathbf{w} \in \mathbb{R}^M$$

$$L(\mathbf{w}) = \sum_{i=1}^N (y_i - \mathbf{w}^T \phi(\mathbf{x}_i))^2 = \|\mathbf{y} - \Phi \mathbf{w}\|_2^2$$

Φ has N rows and M feature columns. Entry (i, j) is $\phi_j(\mathbf{x}_i)$.

$L(\mathbf{w})$ is the RSS criterion. Minimise it over the same coefficient vector.

More equations than unknowns

Design matrix

N rows (examples), M columns (features, including an intercept if used).

When $N > M$

An exact fit may not exist: solve for the smallest residual norm.

Example

30 noisy observations and 2 line parameters.

Uniqueness

The least-squares coefficients are unique when X has full column rank.

Interpolation and overfitting

When $M > N$

The system has more parameters than observations.

With full row rank

Every training target vector can be fitted, with many possible coefficients.

Example

30 observations and 126 polynomial features can interpolate the data.

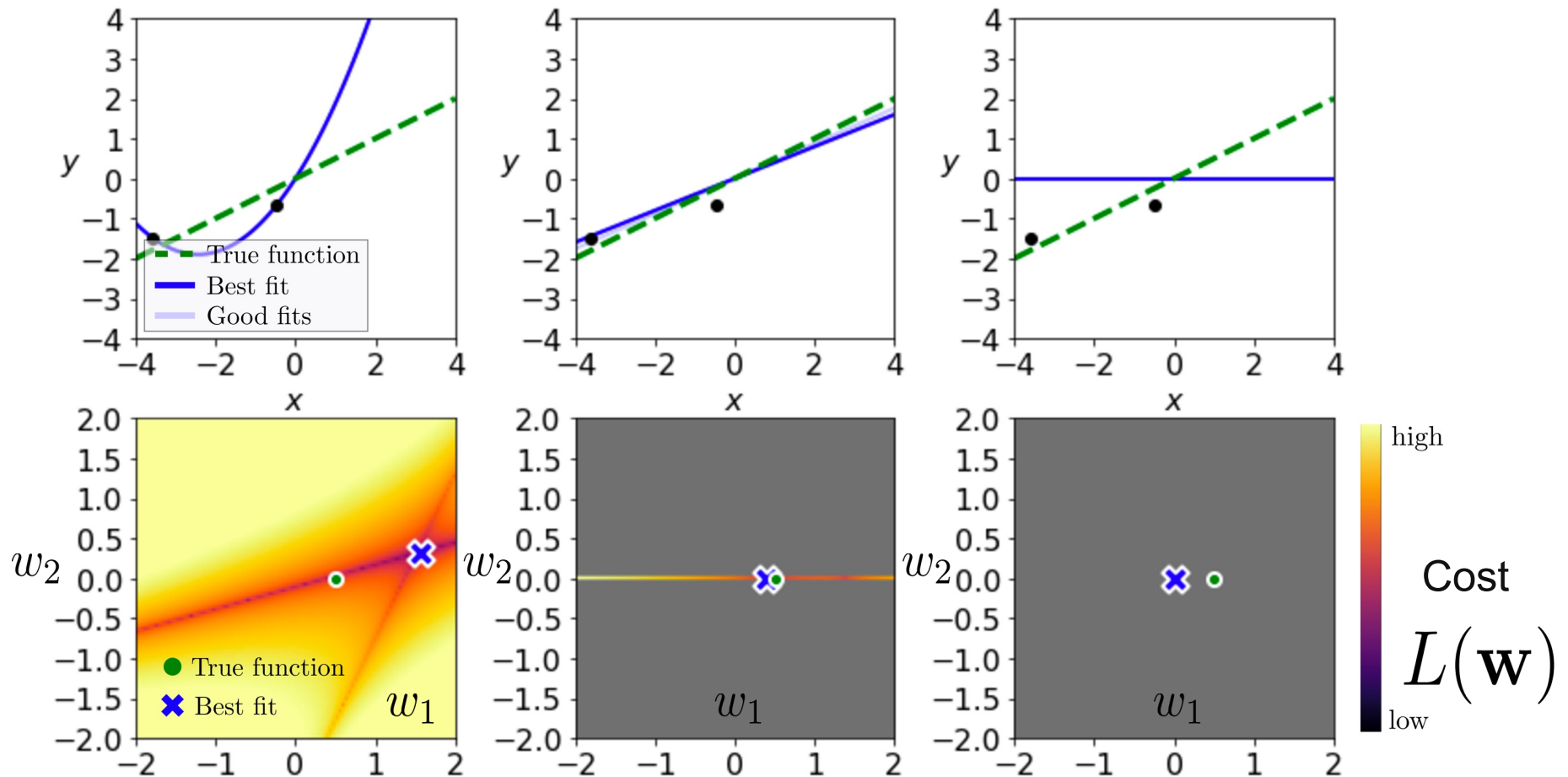
The real question

How well does the chosen solution predict independent examples?

A smaller function class rules out candidates

$$u = w_1 x + w_2 x^2$$

Smaller class from left to right

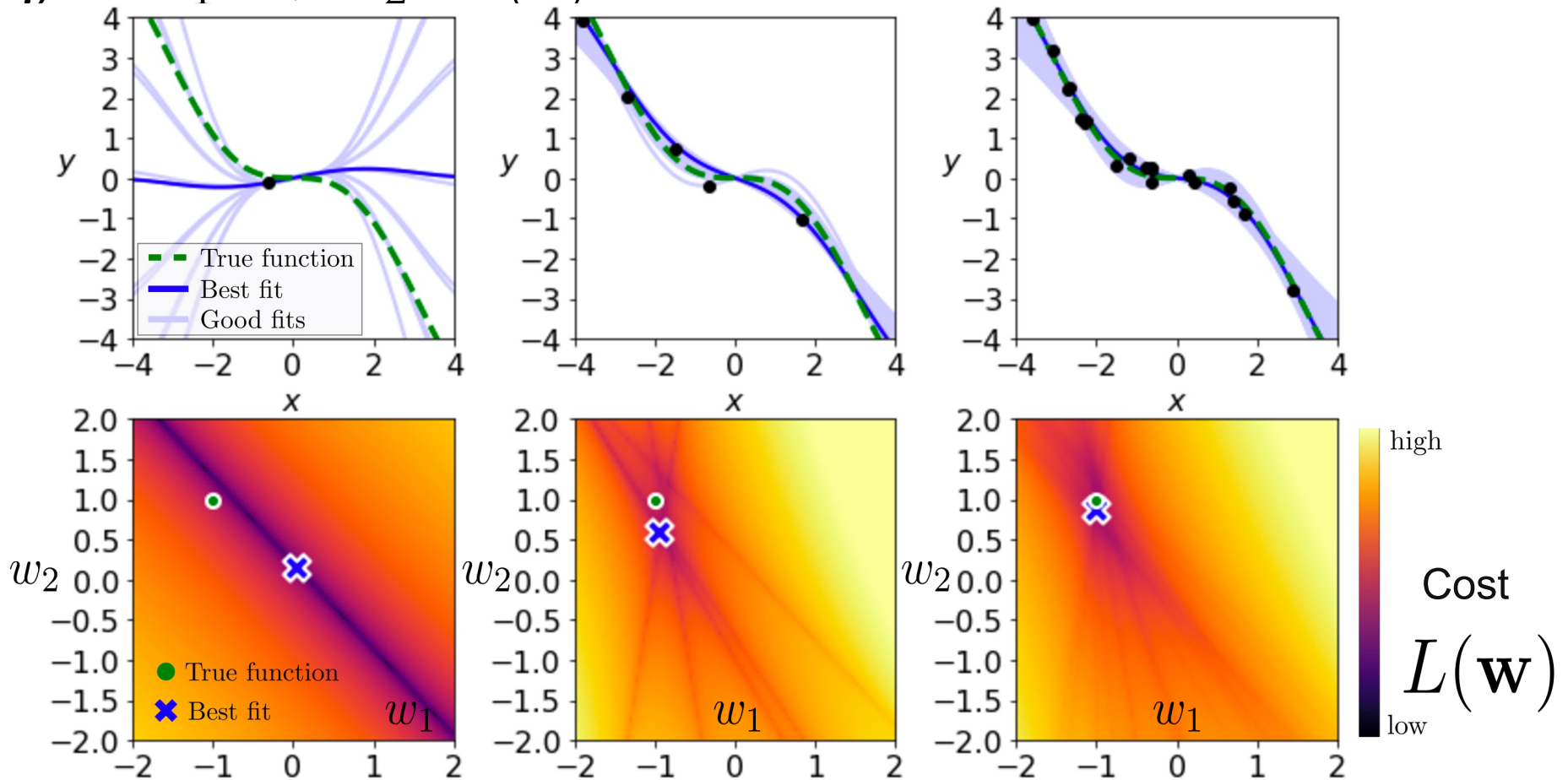


Here the family narrows from quadratic to linear to the zero function.
A restrictive family can exclude the true function.

More observations constrain the fitted function

$$y = w_1 x + w_2 \sin(x)$$

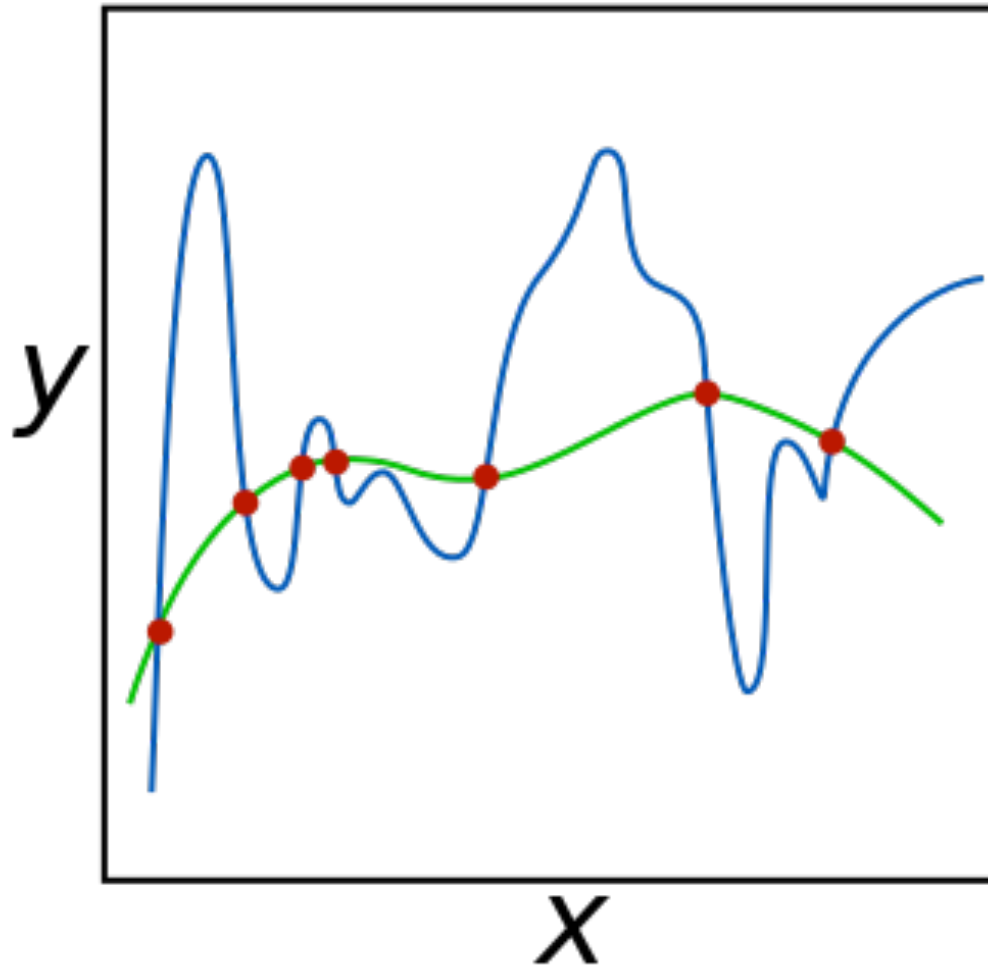
More data from left to right



More observations narrow the range of fits consistent with the data.
The specified function class remains the same.

Avoiding overfitting: regularization

Flexible models can fit noise as well as signal.



Regularisation: a preference over functions

Among functions that fit the data, favour those with lower penalty.

$$\hat{\mathbf{w}} \in \arg \min_{\mathbf{w}} [L(\mathbf{w}) + \lambda \Omega(\mathbf{w})]$$

Ridge uses the squared L2 coefficient penalty

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_2^2$$

$L(\mathbf{w})$ is the residual sum of squares.

$\Omega(\mathbf{w})$ is a chosen penalty on the coefficients.

$\lambda \geq 0$ sets its strength.

Ridge regression

$$L_{\text{ridge}}(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

Fit to the observations

The first term is RSS. The second is the squared L2 coefficient penalty.

Preference for smaller coefficients

$\lambda \geq 0$: ridge strength. Larger values shrink coefficients more.

Centre inputs and targets by subtracting their training means. Reuse those statistics at validation and test time.

Fit the intercept separately, without a penalty.

Ridge: the parameter estimate

$$L_{\text{ridge}}(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

Take the gradient with respect to $\mathbf{w}$

$$\nabla_{\mathbf{w}} L_{\text{ridge}}(\mathbf{w}) = -2X^T(\mathbf{y} - X\mathbf{w}) + 2\lambda\mathbf{w}$$

Set the gradient to zero and rearrange

$$(X^T X + \lambda I) \hat{\mathbf{w}} = X^T \mathbf{y}$$

I is the identity matrix. For $\lambda > 0$, the solution is unique.

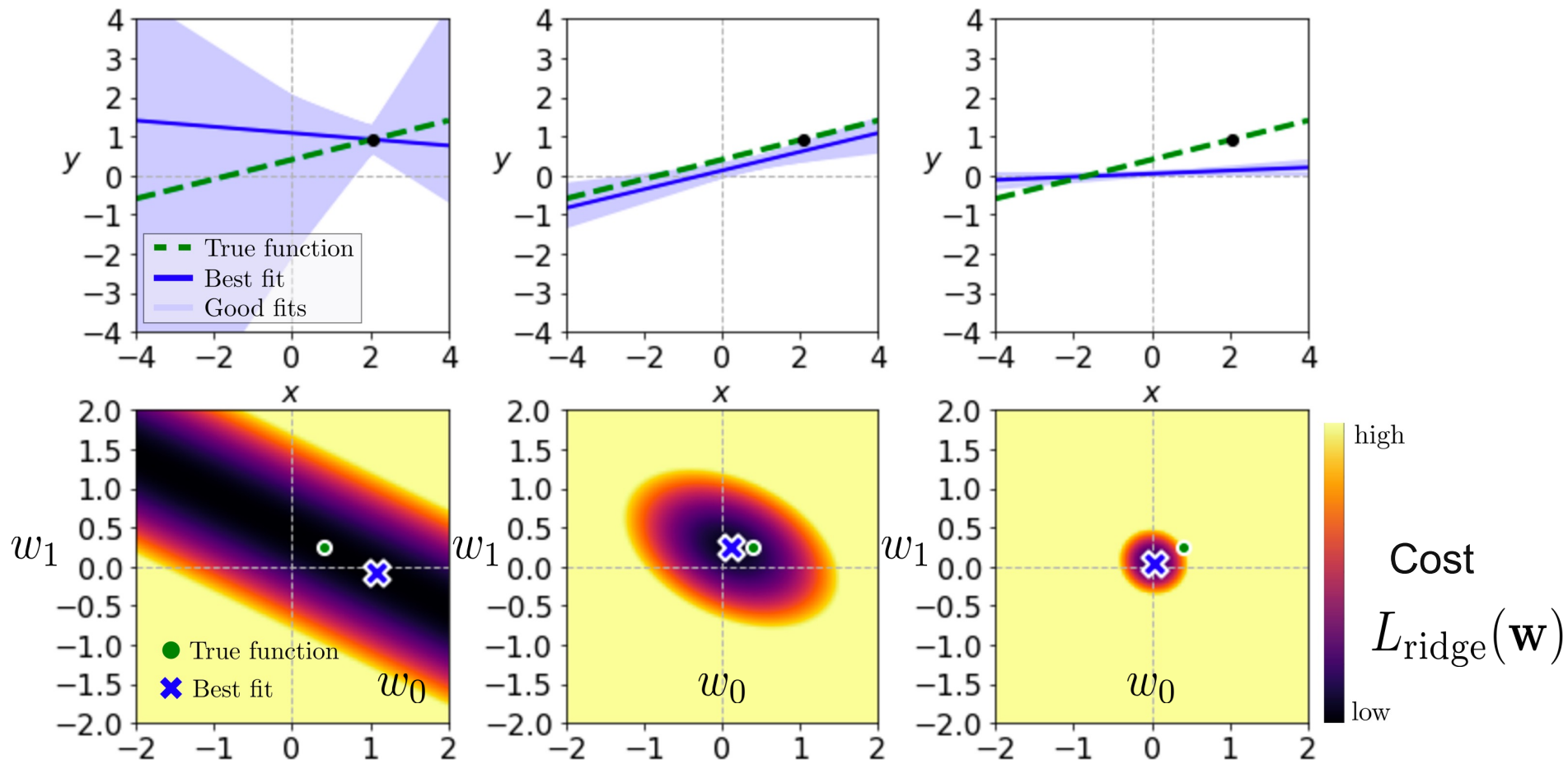
$$\hat{\mathbf{w}} = (X^T X + \lambda I)^{-1} X^T \mathbf{y}$$

In code, solve the linear system.

Stronger regularisation restricts good fits

$$y = w_0 + w_1x$$

Stronger penalty from left to right



This schematic penalises both coefficients. Our regression objective leaves the intercept unpenalised.

Cross-validation and final evaluation

Choose λ using K validation folds

K is the number of folds.

Fit on $K - 1$ folds and score the remaining fold.

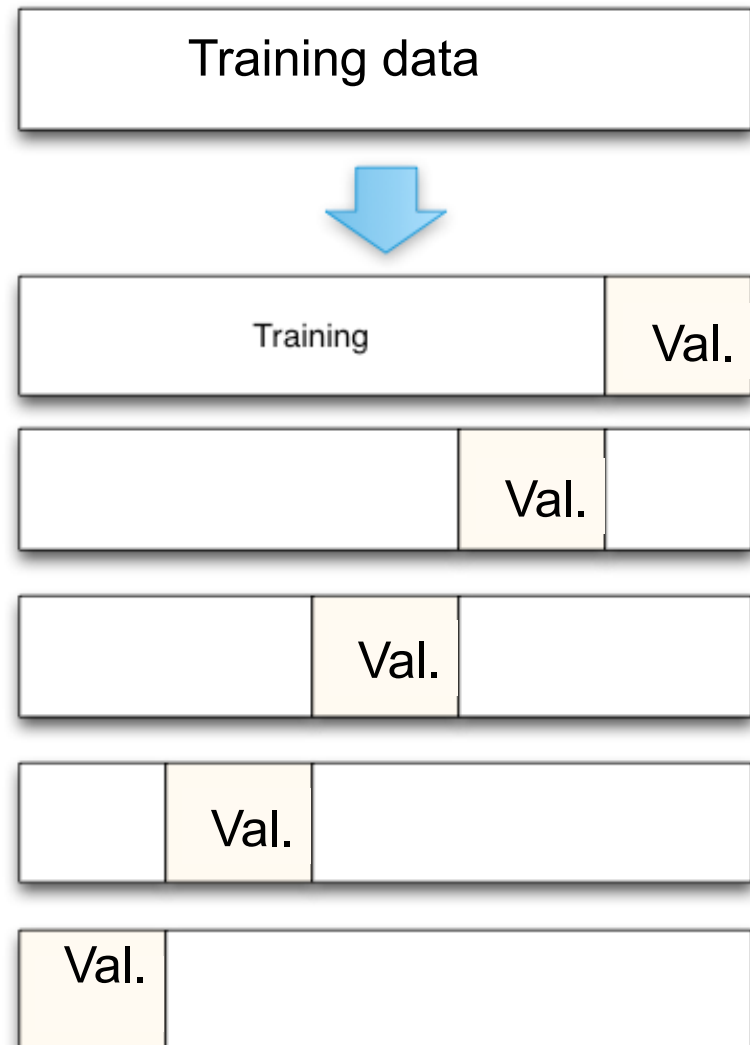
Rotate the held-out fold. Average its scores across all K fits.

Compare candidate λ values using this average.

Then freeze the modelling choices

Refit using all development data.

Evaluate once on the untouched test set.



Cross-validation must match deployment

Independent examples

K is the number of folds. For independent examples, rotate the held-out fold.

Repeated people, devices or sites

Keep related observations in the same fold.

Forecasting

Train on earlier data and validate on later data.

Prevent leakage

Fit scaling, imputation and feature selection inside each training fold.

Leakage through related observations

A repeated-measurement example

Several sensor readings come from each device. Random row splitting places the same devices in training and validation.

The deployment question

Will the model predict another reading from a known device, or a reading from a new device?

A suitable split

For new devices, keep all readings from each device together. For forecasting, train on the past and evaluate later observations.

Three roles for your data

Training

Fit parameters and any learned preprocessing.

Validation

Choose λ and other modelling decisions.

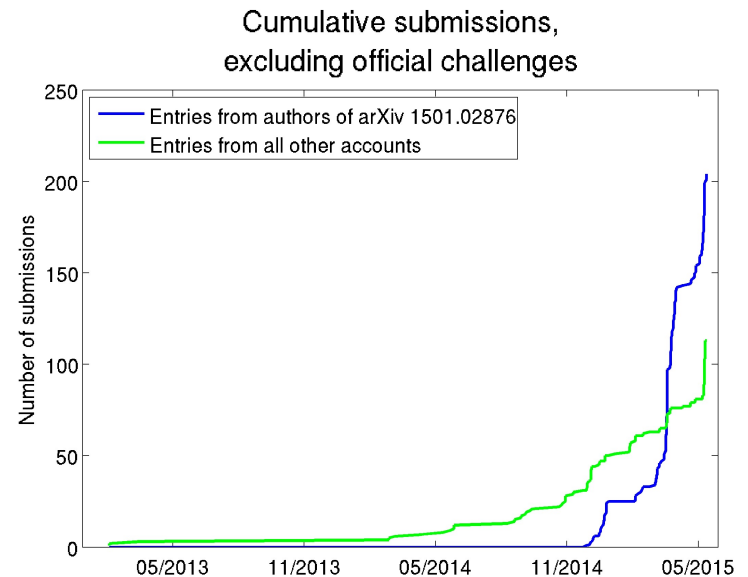
Final test

Assess the model after its choices are fixed.

Repeated test feedback

Changing the model in response to test results uses that set for validation.

Don't do this at work



[Baidu / ImageNet \(2015\): background](#)

Vector norms measure coefficient size

$$\|\mathbf{w}\|_2 = \left(\sum_{j=1}^M w_j^2 \right)^{1/2}$$

Euclidean length. Ridge penalises its square.

$$\|\mathbf{w}\|_1 = \sum_{j=1}^M |w_j|$$

Sum of absolute values. Lasso uses this penalty.

Here $\mathbf{w}$ stacks M penalised coefficients; the intercept is separate.
 j indexes these coefficients.

Lasso: an L1 coefficient penalty

Lasso (least absolute shrinkage and **selection** operator)

$$L_{\text{lasso}}(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_1$$

L_{lasso} : the penalised training objective.

The L1 penalty sums absolute coefficient values.

$$\|\mathbf{w}\|_1 = \sum_{j=1}^M |w_j|$$

Unlike ridge, lasso can set coefficients exactly to zero.
This selects a subset of the available features.

Why does L1 norm yield sparse solutions?

$\mathbf{x}$: observed vector. $\boldsymbol{\alpha}$: approximation. M : coordinates. λ : penalty strength.
The hat marks the minimiser. μ : constraint radius. “s.t.” means subject to.

$$\hat{\boldsymbol{\alpha}}(\lambda) = \arg \min_{\boldsymbol{\alpha} \in \mathbb{R}^M} \left\{ \frac{1}{2} \|\mathbf{x} - \boldsymbol{\alpha}\|_2^2 + \lambda \|\boldsymbol{\alpha}\|_1 \right\}$$

Penalised and constrained forms:

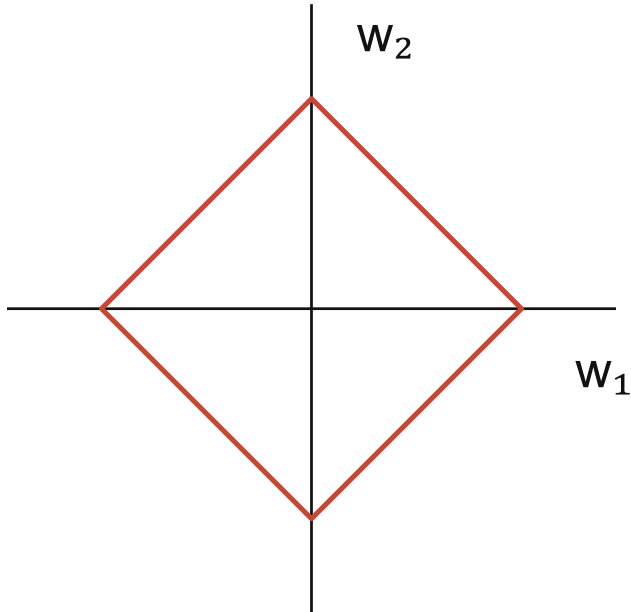
$$\hat{\boldsymbol{\alpha}}(\mu) = \arg \min_{\boldsymbol{\alpha} \in \mathbb{R}^M} \frac{1}{2} \|\mathbf{x} - \boldsymbol{\alpha}\|_2^2 \quad \text{s.t.} \quad \|\boldsymbol{\alpha}\|_1 \leq \mu$$

A matching radius μ gives the same solution for each $\lambda \geq 0$.
The geometry explains why an L1 penalty can produce zeros.

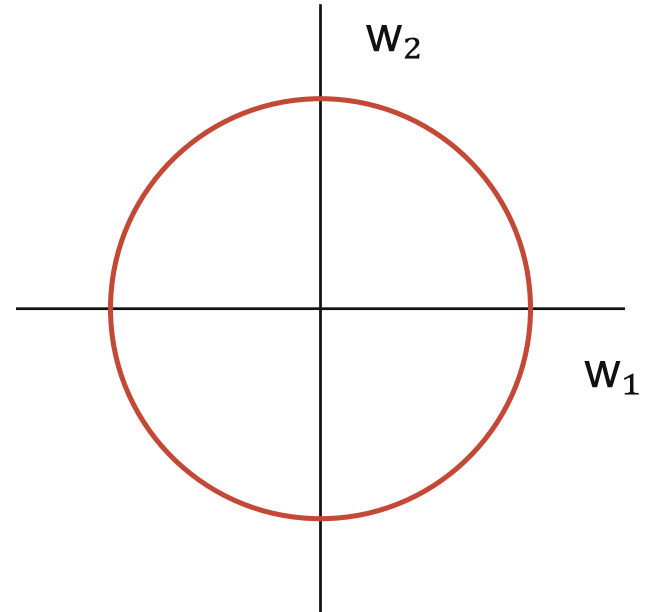
Vector norms determine geometry

A unit ball contains vectors with norm at most one. w_1 , w_2 are coefficients.

L1 ball

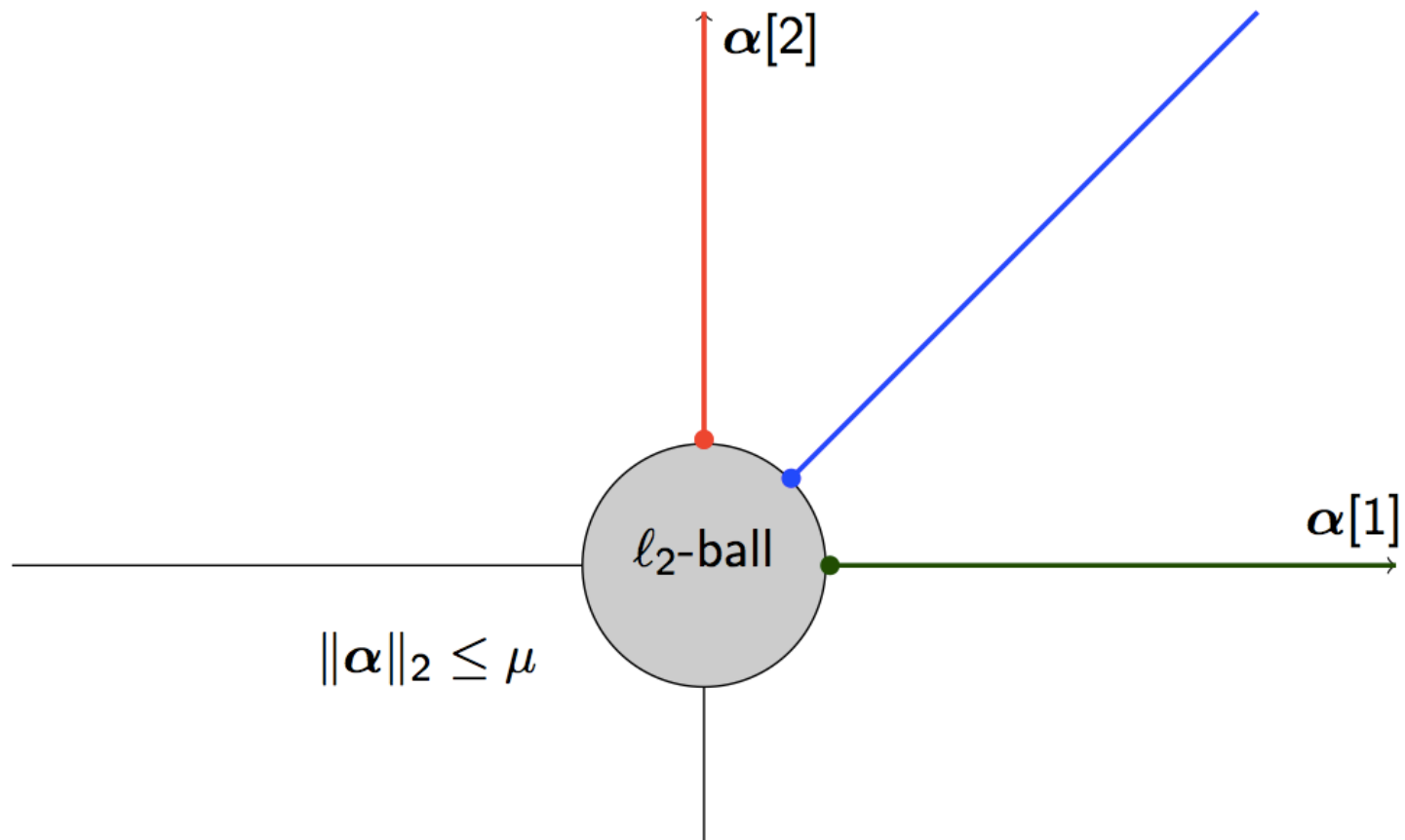


L2 ball



The L1 boundary has corners. The L2 boundary is smooth.
Those corners can produce exactly zero coefficients (sparsity).

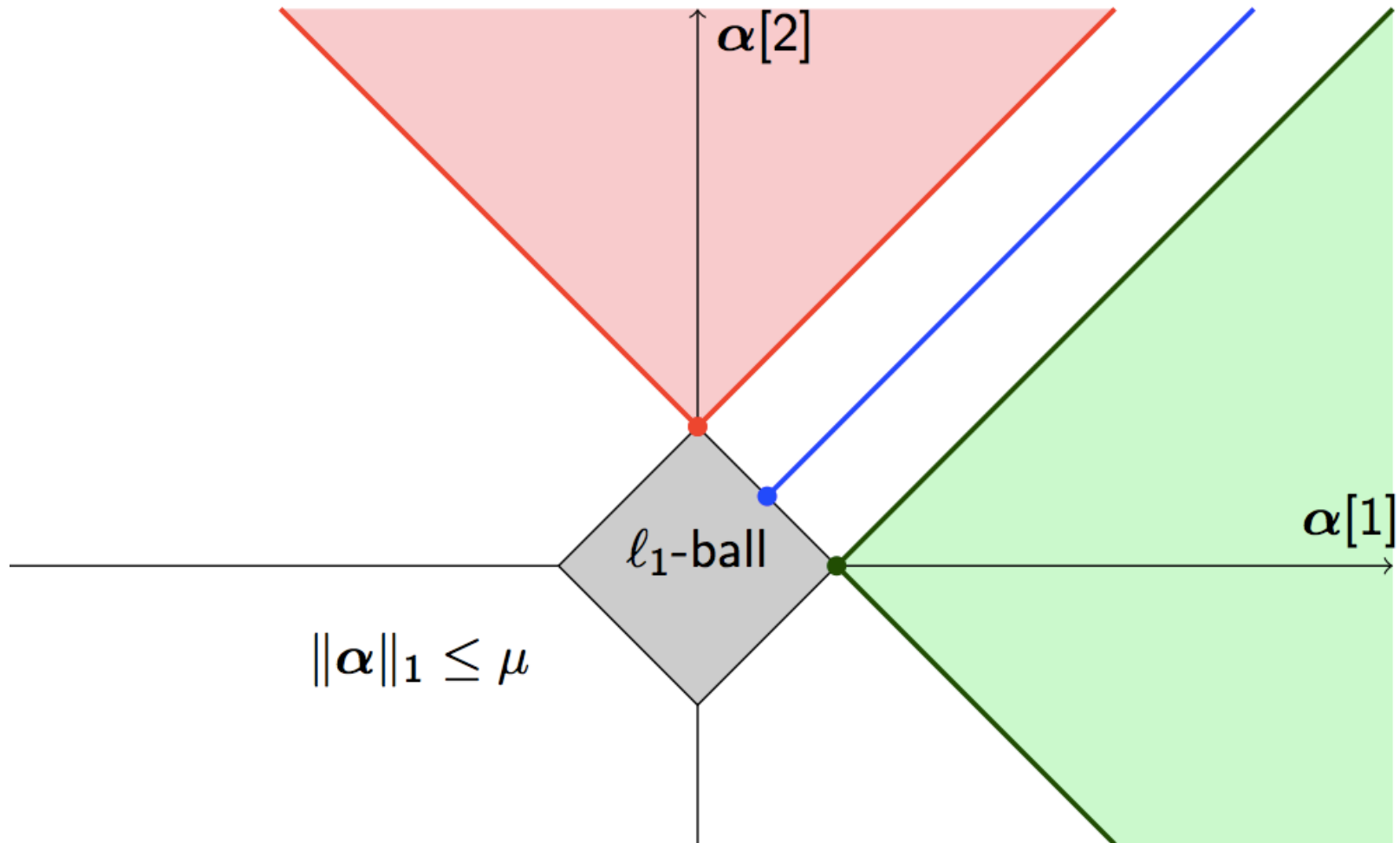
L2 geometry: projection onto a ball



The ℓ_2 -norm is isotropic.

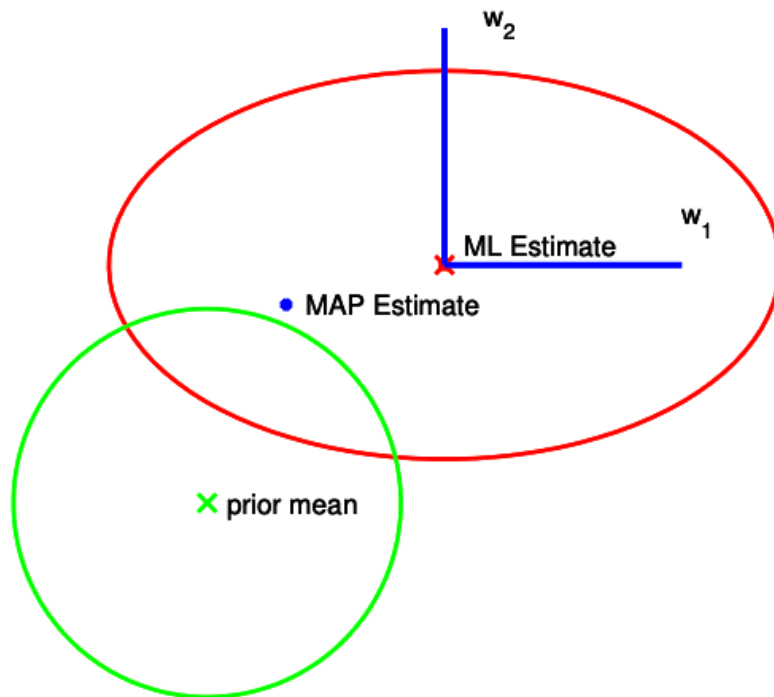
The nearest point usually has several nonzero coordinates.

L1 geometry: projection onto corners



Many observations project onto a corner, where one coordinate is zero.

Probabilistic fitting and regularisation



Likelihood

Scores how well parameters explain the observed data.

Prior

Describes parameter beliefs before these observations.

Posterior

Combines likelihood and prior after observing the data.

w_1, w_2 : two coefficients. ML: maximum likelihood.

MAP: maximum a posteriori. A prior gives a probabilistic basis for a penalty.

Random variables and observed values

Y is a random variable; y is one observed value.

Example: choose a box, then a fruit

Y records the fruit: apple (a) or orange (o).

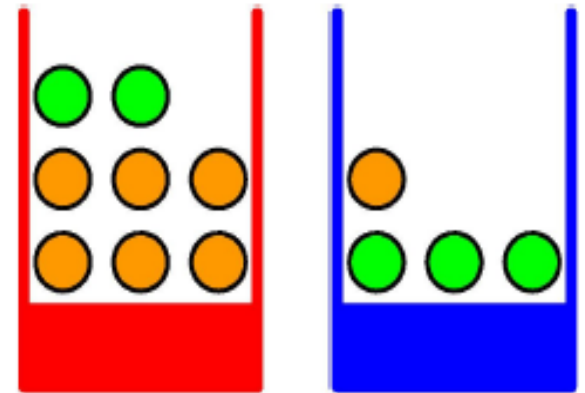
B records the box: red (r) or blue (b).

$$Y \in \{a, o\}, \quad y = a \text{ or } y = o$$

$$p(y) = p(Y = y) = \Pr(Y = y)$$

$$p(y) \geq 0, \quad \sum_y p(y) = 1$$

$p(y)$ and $\Pr(Y = y)$ denote probability. PMF: probability mass function.



Continuous variables: density and probability

A density gives probability through area under its curve.

$$\Pr(a < Y \leq b) = \int_a^b p(y) \, dy$$

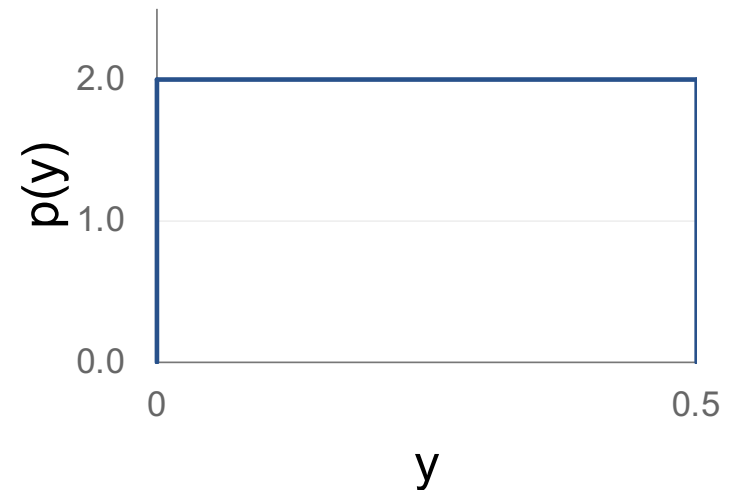
$$p(y) \geq 0, \quad \int_{-\infty}^{\infty} p(y) \, dy = 1$$

a and b are interval endpoints.

$\int$ denotes integration (area).

$$\int_0^{0.5} 2 \, dy = 2 \times 0.5 = 1$$

For a continuous density, $\Pr(Y = y) = 0$.



Uniform on $[0, 0.5]$: density 2.

Zero outside this interval.

$$\Pr(0.1 < Y < 0.3) = 0.4$$

Sampling assumptions behind the likelihood

Earlier: the training set is a finite sample of a population.

Identically distributed

Each pair follows the same population distribution.

Independent

One sampled pair does not affect another sampled pair.

$$p(\mathcal{D}) = \prod_{i=1}^N p(\mathbf{x}_i, y_i)$$

IID combines both assumptions. $p(\mathcal{D})$ denotes the joint mass or density of the observed dataset. $\prod$ multiplies over its N examples.

Joint, marginal and conditional probability

Choose red with probability 0.4; blue with probability 0.6.

Joint probability	Red box	Blue box	Fruit total
Apple	0.10	0.45	0.55
Orange	0.30	0.15	0.45
Box total	0.40	0.60	1.00

$$p(B = r, Y = a) = 0.10$$

$$p(Y = a) = 0.10 + 0.45 = 0.55$$

$$p(Y = a \mid B = r) = \frac{0.10}{0.40} = 0.25$$

The bar | means “given”. A comma joins events. Marginals sum over the other variable.

Bayes' rule: reverse the conditioning

Observed: an apple. Here b and b' range over both possible box colours.

$$p(B = b \mid Y = y) = \frac{p(Y = y \mid B = b) p(B = b)}{\sum_{b'} p(Y = y \mid B = b') p(B = b')}$$

$$\text{posterior} = \frac{\text{likelihood} \times \text{prior}}{\text{evidence}}$$

$$p(B = r \mid Y = a) = \frac{0.25 \times 0.40}{0.55} = \frac{2}{11} \simeq 0.182$$

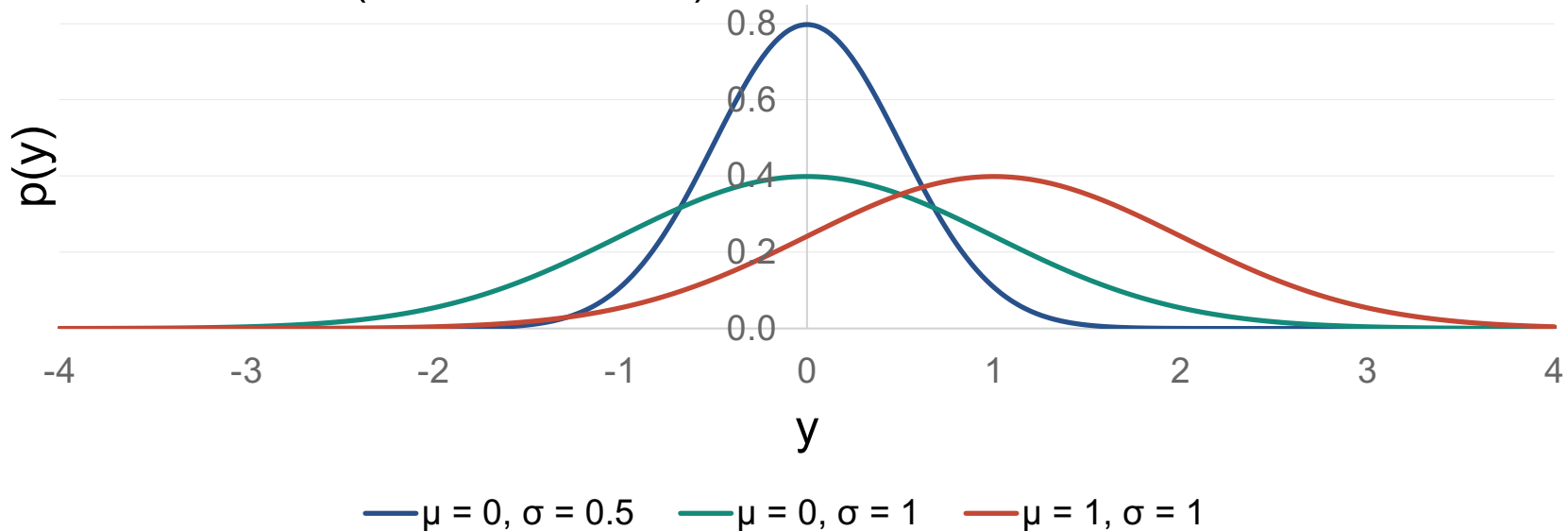
Prior: box probabilities before seeing the fruit. Posterior: after seeing it.

Likelihood: fruit probability for a given box. Evidence: total fruit probability.
For continuous unknowns, use densities and integrate in the denominator.

The one-dimensional Gaussian distribution

$$Y \sim \mathcal{N}(\mu, \sigma^2), \quad p(y) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{(y - \mu)^2}{2\sigma^2}\right]$$

$\sim$: distributed as. $\mathcal{N}(\text{mean}, \text{variance})$: Gaussian distribution.



$$\mu = E[Y], \quad \sigma^2 = E[(Y - \mu)^2]$$

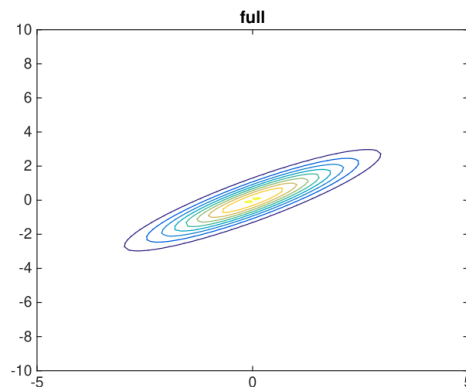
$E[\cdot]$: expectation. $\exp(t) = e^t$. π : circle constant. $\sqrt{\cdot}$: square root.

μ is the mean. σ is the standard deviation. The variance is σ^2 .

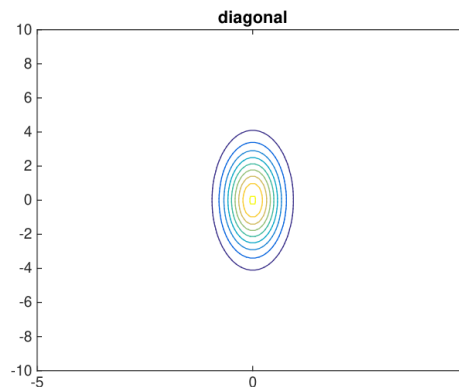
The multivariate Gaussian distribution

$$\mathbf{Y} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma), \quad \boldsymbol{\mu} = \mathbb{E}[\mathbf{Y}], \quad \Sigma = \text{Cov}(\mathbf{Y})$$

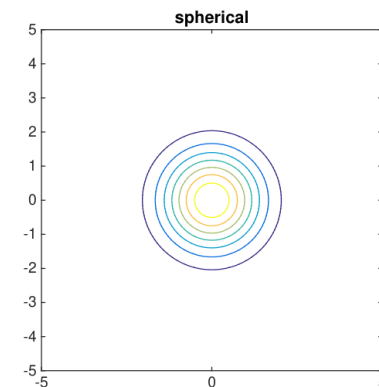
$$p(\mathbf{y}) = \frac{\exp\left[-\frac{1}{2}(\mathbf{y} - \boldsymbol{\mu})^T \Sigma^{-1}(\mathbf{y} - \boldsymbol{\mu})\right]}{(2\pi)^{d/2} |\Sigma|^{1/2}}$$



(a)



(b)



(c)

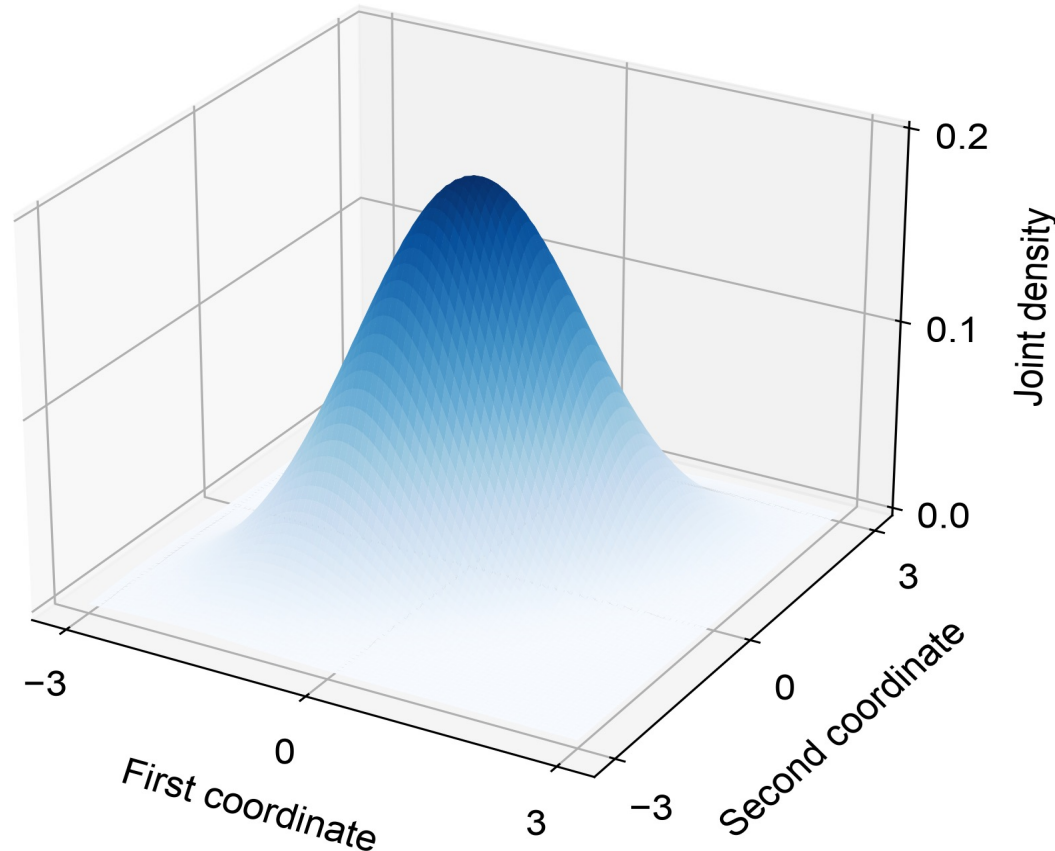
d : output coordinates. $\boldsymbol{\mu}$: mean vector. $\Sigma = \text{Cov}(\mathbf{Y})$: covariance matrix.

$|\Sigma|$: determinant. Σ^{-1} : inverse. Cov means covariance.

Diagonal entries: variances. Off-diagonal entries: covariances.

Positive definite: variance > 0 along any direction.

A Gaussian density in two dimensions



Two coordinates describe one outcome.

The mean sets the centre.
Covariance controls spread and orientation.

Probability is volume under the surface over a region of the plane.

Diagonal covariance and sums of squares

$$\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_d^2)$$

diag forms a diagonal matrix. σ_j^2 is coordinate j 's variance.

$$(\mathbf{y} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{y} - \boldsymbol{\mu}) = \sum_{j=1}^d \frac{(y_j - \mu_j)^2}{\sigma_j^2}$$

Fixed diagonal variances give a variance-weighted sum of squares.

$$\Sigma = \sigma^2 \mathbf{I} \quad \Rightarrow \quad \frac{1}{\sigma^2} \sum_{j=1}^d (y_j - \mu_j)^2 = \frac{1}{\sigma^2} \|\mathbf{y} - \boldsymbol{\mu}\|_2^2$$

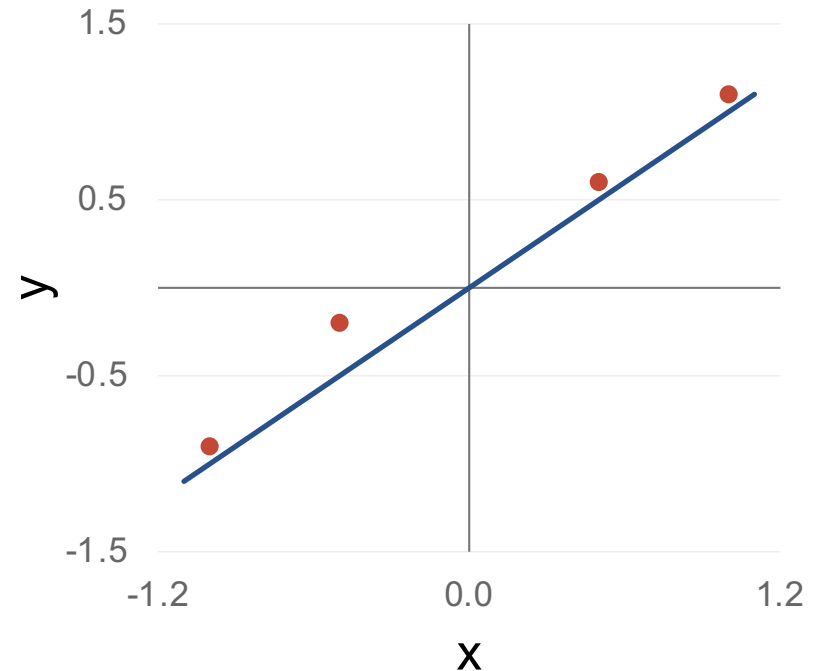
Fixed equal variances give ordinary least squares, up to scale.

A probabilistic model for regression

$$y_i = \mathbf{x}_i^T \mathbf{w} + \epsilon_i$$

$$\epsilon_i \sim N(0, \sigma^2)$$

$$y_i \mid \mathbf{x}_i, \mathbf{w} \sim N(\mathbf{x}_i^T \mathbf{w}, \sigma^2)$$



ϵ : Gaussian observation noise. $\mathbf{w}$ sets the mean. σ^2 is the noise variance.

Residual after fitting: $y_i - \mathbf{x}_i^T \hat{\mathbf{w}}$ The hat on $\mathbf{w}$ marks fitted coefficients.

Likelihood of the observed targets

Condition on the inputs

Independent Gaussian noise makes the likelihood a product over examples. $N(y; \text{mean}, \text{variance})$ denotes a density evaluated at y .

$$\begin{aligned} p(\mathbf{y} \mid X, \mathbf{w}, \sigma^2) &= \prod_{i=1}^N \mathcal{N}(y_i; \mathbf{x}_i^T \mathbf{w}, \sigma^2) \\ &= (2\pi\sigma^2)^{-N/2} \exp \left[-\frac{\|\mathbf{y} - X\mathbf{w}\|^2}{2\sigma^2} \right] \end{aligned}$$

Maximum likelihood

Choose the parameters that give the observed targets the largest density.

Gaussian likelihood gives least squares

$$\begin{aligned} & -\log p(\mathbf{y} \mid X, \mathbf{w}, \sigma^2) \\ &= \frac{N}{2} \log(2\pi\sigma^2) + \frac{\|\mathbf{y} - X\mathbf{w}\|^2}{2\sigma^2} \end{aligned}$$

$\log$ denotes the natural logarithm. For fixed σ^2 :

ML: maximum likelihood. For fixed σ^2 , the constant and positive scale do not change the minimiser in $\mathbf{w}$.

$$\hat{\mathbf{w}}_{\text{ML}} = \arg \min_{\mathbf{w}} \|\mathbf{y} - X\mathbf{w}\|^2$$

Next: a Gaussian prior explains the ridge penalty.

A Gaussian prior on the coefficients

$$\mathbf{y} \mid X, \mathbf{w} \sim \mathcal{N}(X\mathbf{w}, \sigma^2 I)$$

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2 I)$$

τ^2 : prior coefficient variance. $\propto$ means “proportional to”.

σ^2 measures observation noise. τ^2 controls prior coefficient spread.

The posterior is proportional to likelihood times prior.

$$p(\mathbf{w} \mid X, \mathbf{y}) \propto p(\mathbf{y} \mid X, \mathbf{w}) p(\mathbf{w})$$

Centred data. The intercept is handled separately and is not penalised.

MAP: maximum a posteriori estimation

Combine the likelihood of the data with a prior on the parameters.

$$p(\mathbf{w} \mid \mathbf{X}, \mathbf{y}) \propto p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) p(\mathbf{w})$$

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg \max_{\mathbf{w}} p(\mathbf{w} \mid \mathbf{X}, \mathbf{y})$$

$\arg \max$ selects the largest posterior density. The subscript MAP labels this estimate.

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg \min_{\mathbf{w}} \left[-\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) - \log p(\mathbf{w}) \right]$$

MAP estimation gives the ridge objective

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg \min_{\mathbf{w}} \left\{ -\log p(\mathbf{y} \mid X, \mathbf{w}) - \log p(\mathbf{w}) \right\}$$

$$= \arg \min_{\mathbf{w}} \left\{ \frac{\|\mathbf{y} - X\mathbf{w}\|^2}{2\sigma^2} + \frac{\|\mathbf{w}\|^2}{2\tau^2} \right\}$$

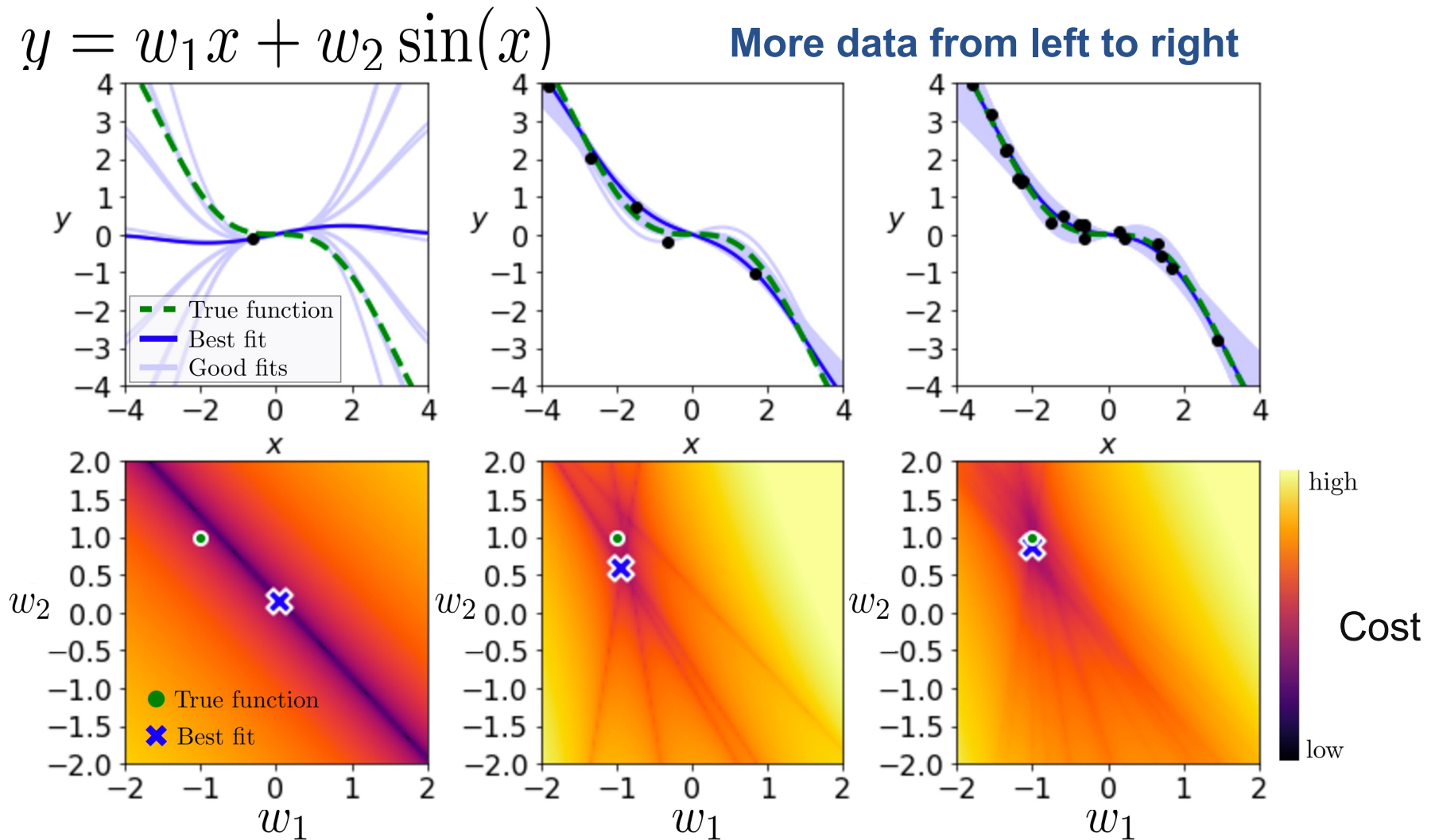
$$= \arg \min_{\mathbf{w}} \left\{ \|\mathbf{y} - X\mathbf{w}\|^2 + \frac{\sigma^2}{\tau^2} \|\mathbf{w}\|^2 \right\}$$

$$\lambda = \frac{\sigma^2}{\tau^2}$$

For mean squared error (MSE = RSS / N),
use:

$$\lambda = \frac{\sigma^2}{N\tau^2}$$

More observations constrain the fitted function



More observations narrow the range of fits consistent with the data.
The specified function class remains the same.

Posterior uncertainty about the parameters

$$p(\mathbf{w} \mid X, \mathbf{y}) = N(\mathbf{m}, S)$$

$$S = \left(\frac{X^T X}{\sigma^2} + \frac{I}{\tau^2} \right)^{-1}$$

$$\mathbf{m} = \frac{S X^T \mathbf{y}}{\sigma^2} = (X^T X + \lambda I)^{-1} X^T \mathbf{y}$$

One-coefficient example

$$\sigma^2 = 0.25, \quad \tau^2 = 1, \quad \sum_i x_i^2 = 2.5, \quad \sum_i x_i y_i = 2.4$$

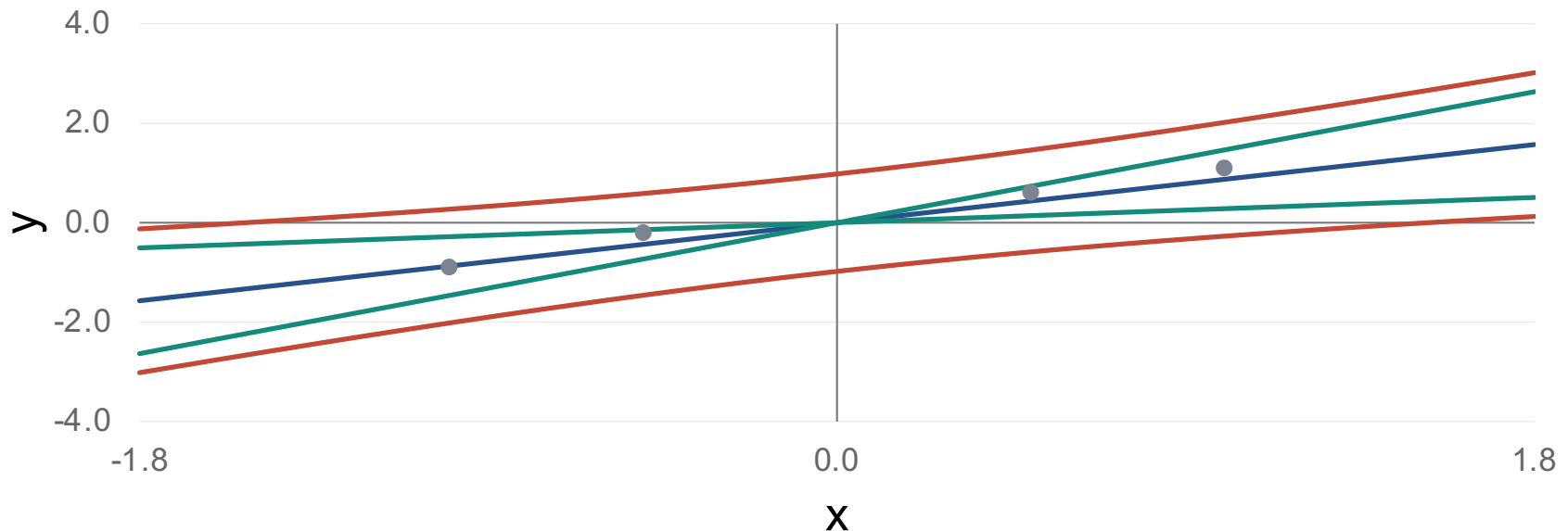
$$w \mid \mathcal{D} \sim N(0.873, 0.091)$$

0.091 is the posterior variance. The posterior standard deviation is about 0.302.

Mean uncertainty and observation noise

$$\text{Var}(f_* \mid \mathcal{D}) = \mathbf{x}_*^T S \mathbf{x}_*$$

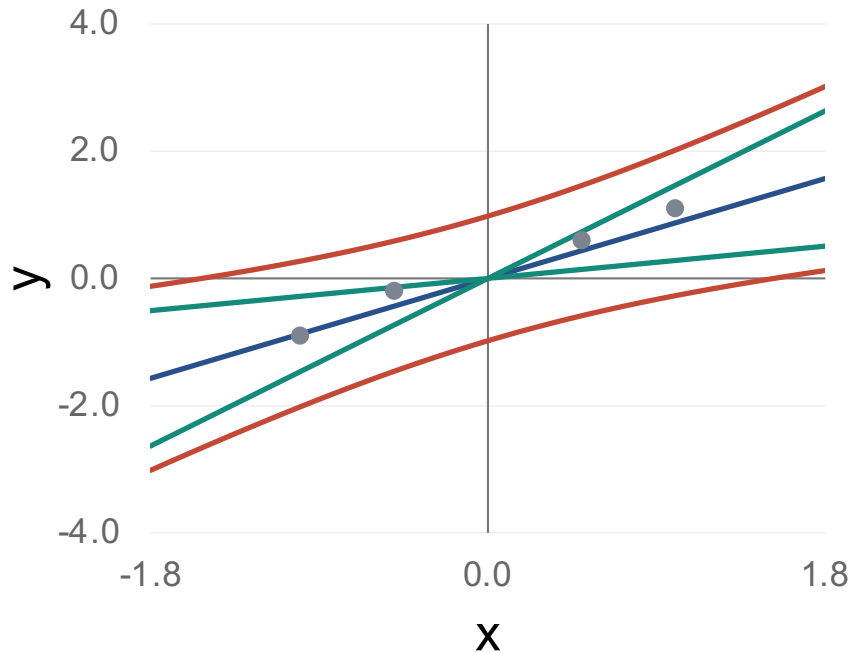
$$\text{Var}(y_* \mid \mathcal{D}) = \mathbf{x}_*^T S \mathbf{x}_* + \sigma^2$$



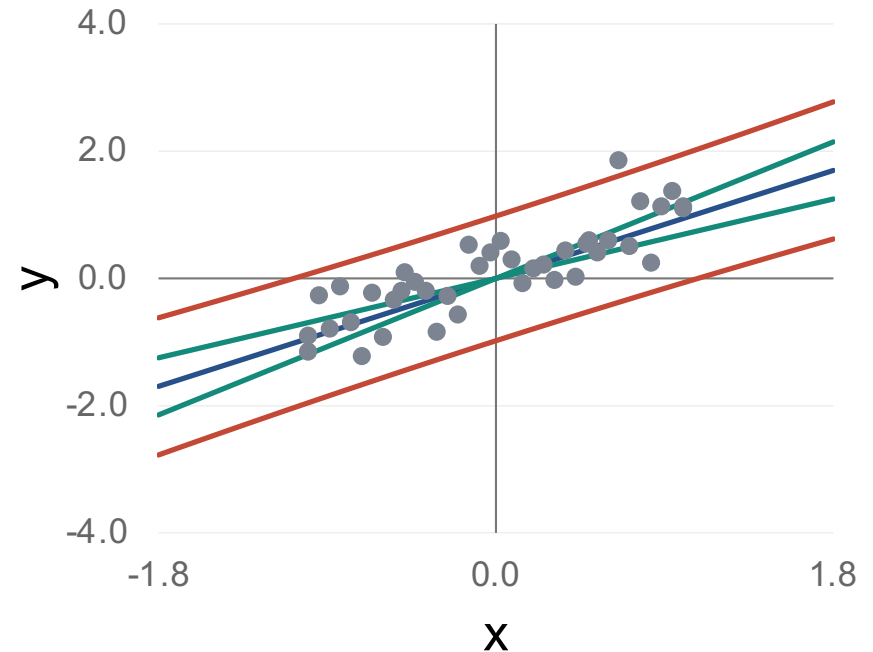
One coefficient, known zero intercept. Green: mean bounds. Red: new-observation bounds (95%).

More data reduces parameter uncertainty

N = 4



N = 40



The green bounds narrow as the coefficient becomes better known.

The red bounds retain the contribution of observation noise.

Training, inference and generalisation

Training

→ Use examples and an objective to choose model parameters.

Inference

→ Apply the trained model to a new input.

Generalisation

→ Perform well on unseen cases from the real world.

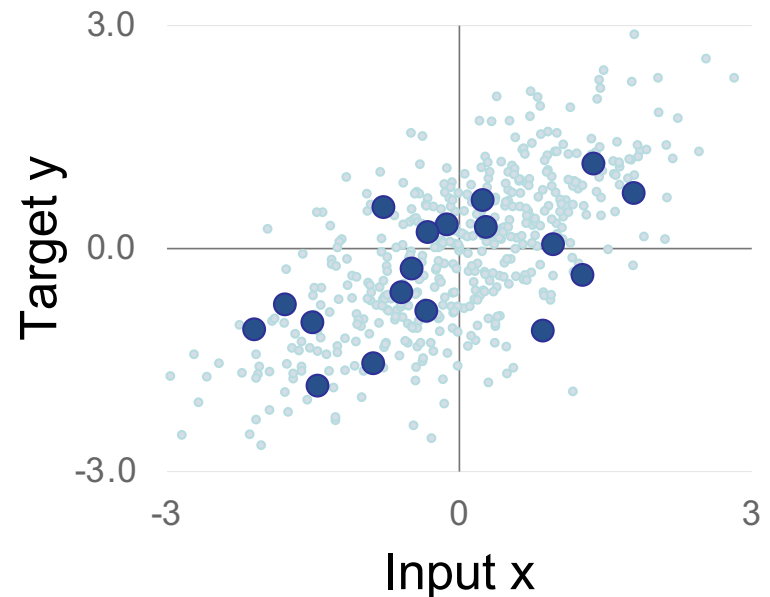
Training data are a sample of the real world

The training sample is finite.

We want low population cost on fresh examples.

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$$

$$(\mathbf{X}_i, Y_i) \sim P, \quad i = 1, \dots, N$$

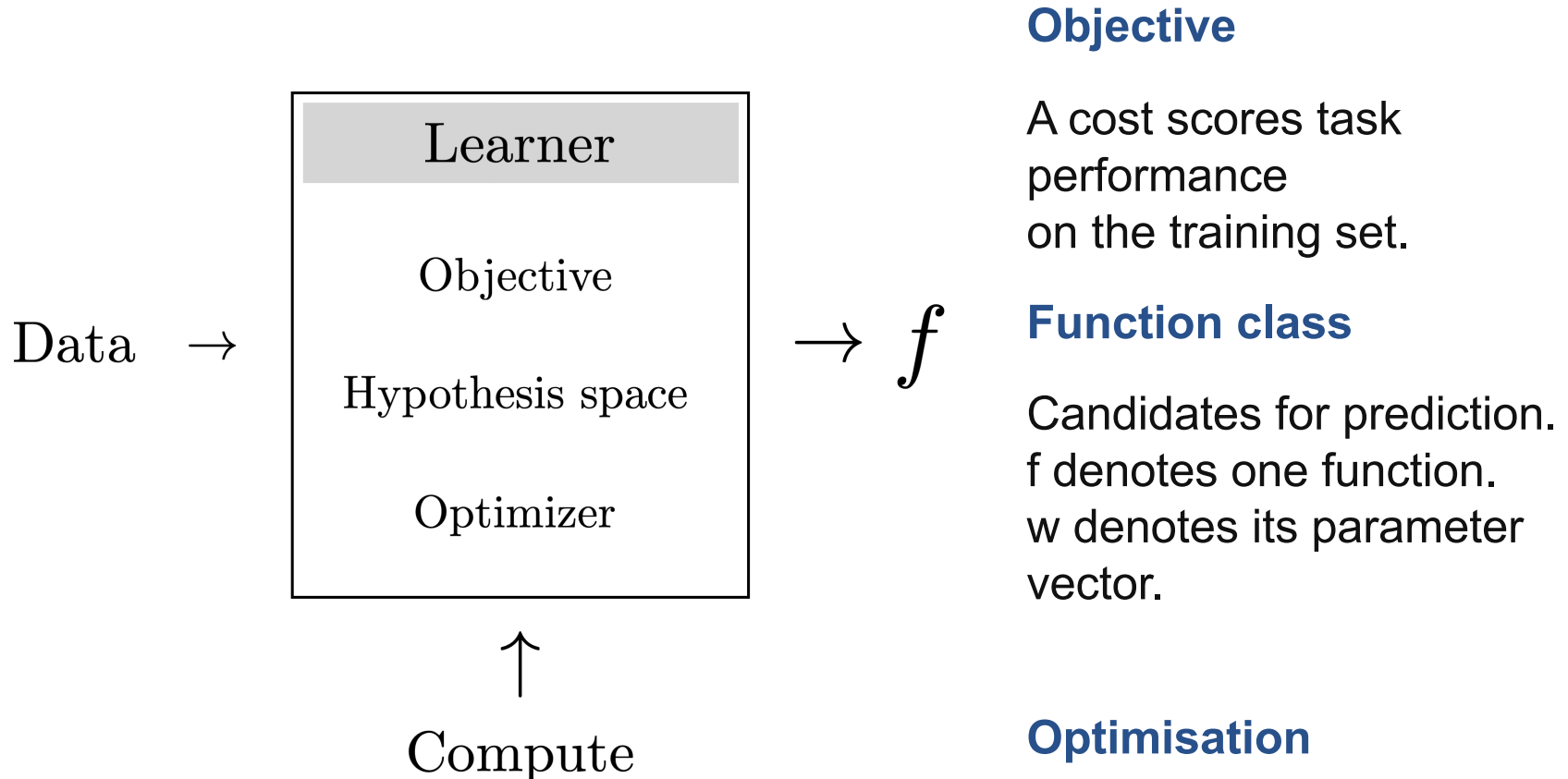


$\mathcal{D}$: training dataset. N : example count. x_i, y_i : observed input and target.

P : population distribution. Upper-case X_i, Y_i denote random draws.

Lower-case x_i, y_i are their observed values.

Choosing a function by minimising cost



A reward measures desirability and is maximised.

Empirical risk minimisation

$\ell(y, \hat{y})$ is a loss that scores one prediction.
Average it over the observed training dataset $\mathcal{D}$.

$$\hat{R}_{\text{train}}(f) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, f(\mathbf{x}_i))$$

$$\hat{f}_{\mathcal{D}} \in \arg \min_{f \in \mathcal{F}} \hat{R}_{\text{train}}(f)$$

$\mathcal{F}$ is the candidate function class. $\arg \min$ selects a minimiser.
 $\hat{f}$ with subscript $\mathcal{D}$ is the predictor fitted using dataset $\mathcal{D}$.

Our goal is low loss on fresh examples from the same population.

Training risk and population risk

$$R(f) = \mathbb{E}_{(X,Y) \sim P} [\ell(Y, f(X))]$$

P is the population distribution. E averages over a fresh pair (X, Y).
R(f) is its expected loss. Training risk averages over the finite dataset.

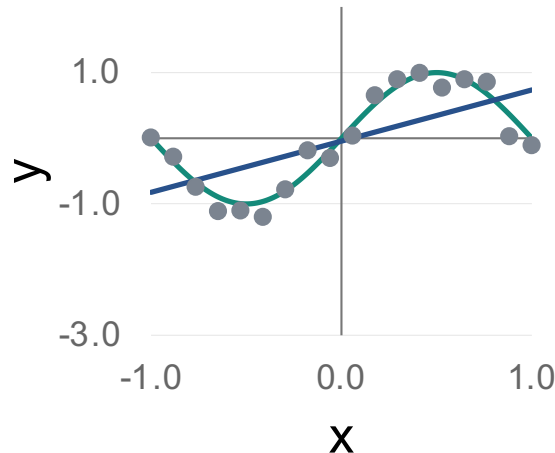
For regression, choose squared loss

$$\ell(y, \hat{y}) = (y - \hat{y})^2, \quad \hat{R}_{\text{train}} = \frac{L(\mathbf{w})}{N}$$

Training mean squared error (MSE) is RSS divided by N.
A small training MSE can coexist with large error on unseen data.

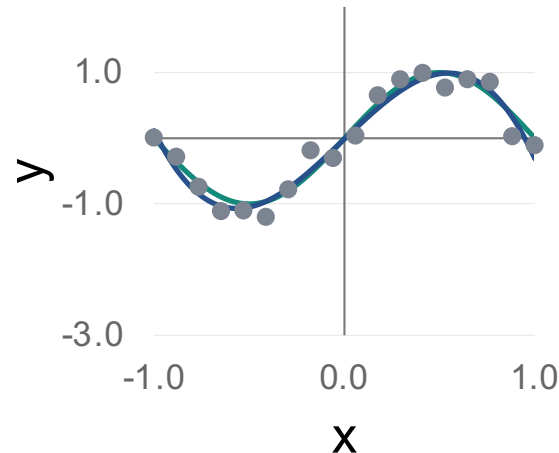
Training fit and unseen predictions

Degree 1



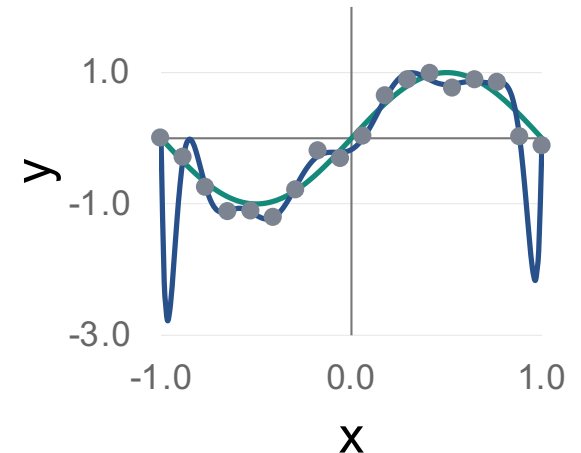
Train MSE: 0.305
Valid. MSE: 0.245

Degree 3



Train MSE: 0.029
Valid. MSE: 0.057

Degree 14



Train MSE: 0.002
Valid. MSE: 0.485

$$\hat{R}_{\text{val}}(\hat{f}_{\mathcal{D}}) = \frac{1}{N_{\text{val}}} \sum_{i=1}^{N_{\text{val}}} (y_i^{\text{val}} - \hat{f}_{\mathcal{D}}(\mathbf{x}_i^{\text{val}}))^2$$

N_{val} counts validation pairs. “val” labels the split.

Same training data; 200 validation pairs. Green is the true mean.

Sources of prediction error

Approximation: the model family is too restrictive

A straight line cannot represent a quadratic mean. Add suitable features.

Estimation: the training sample is finite

Different noisy samples give different fits. More data or regularisation can help.

Optimisation: fitting stops short of its objective

The solver has not reached the empirical minimum. Improve or continue optimisation.

Even the correct mean cannot predict fresh measurement noise.

Expected squared prediction error

$$m(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)], \quad f^*(x) = \mathbb{E}[Y \mid x]$$

$m(x)$: mean fit. $f^*(x)$: true mean. * marks truth. $\mathbb{E}_{\mathcal{D}}$ averages datasets.

$$\begin{aligned} & \mathbb{E}_{\mathcal{D}, Y} \left[(Y - \hat{f}_{\mathcal{D}}(x))^2 \mid x \right] \\ &= \underbrace{(m(x) - f^*(x))^2}_{\text{squared bias}} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - m(x))^2]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}} \end{aligned}$$

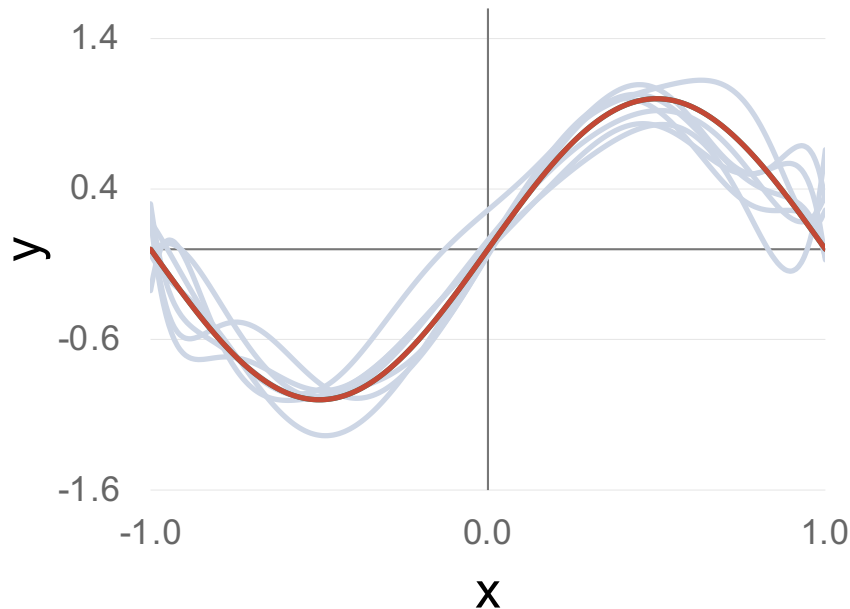
At x = 0.65	Bias ²	Variance	Noise	Total
$\lambda = 0.0001$	0.0000	0.0308	0.0625	0.0933
$\lambda = 10$	0.3772	0.0045	0.0625	0.4442

At fixed x, average over repeated datasets and fresh noise.

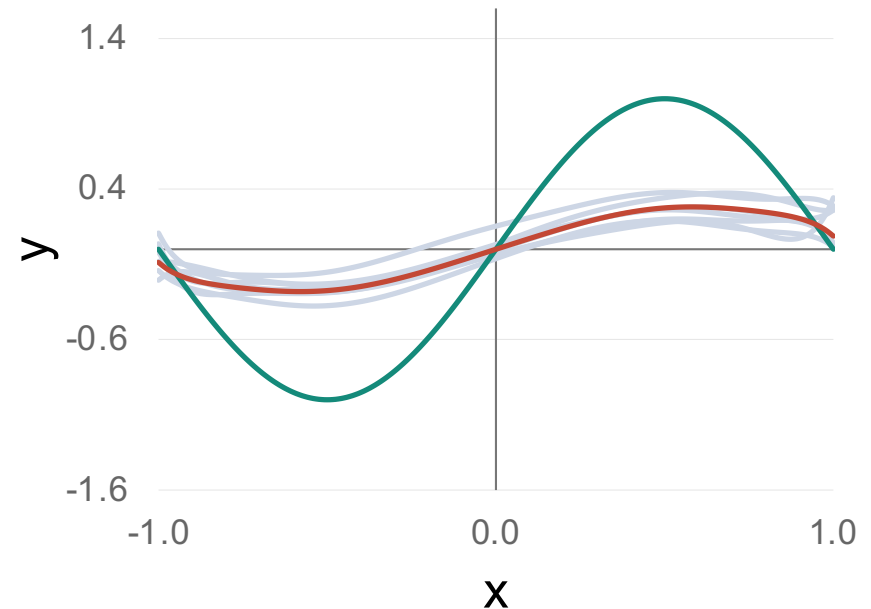
σ^2 is noise variance. The table uses the following ridge experiment.

Bias and variance across repeated datasets

$\lambda = 0.0001$



$\lambda = 10$



Grey: fits from separate datasets. Red: mean fit. Green: generating mean.

Bias: mean fit minus truth. Variance: variation of fits around their mean.
Strong shrinkage reduces variation but can shift the mean fit.

One ridge experiment, two noise levels

$$y_i = \sin(\pi x_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Same 18 inputs, equally spaced over $[-1, 1]$.

Same mean function, $\sin(\pi x)$. ϵ_i is independent Gaussian noise.

σ is its standard deviation. Only σ changes between the plots.

Same function class and fitting objective

$$J(\mathbf{w}) = \|\mathbf{y} - \Phi \mathbf{w}\|_2^2 + \lambda \sum_{j=1}^9 w_j^2$$

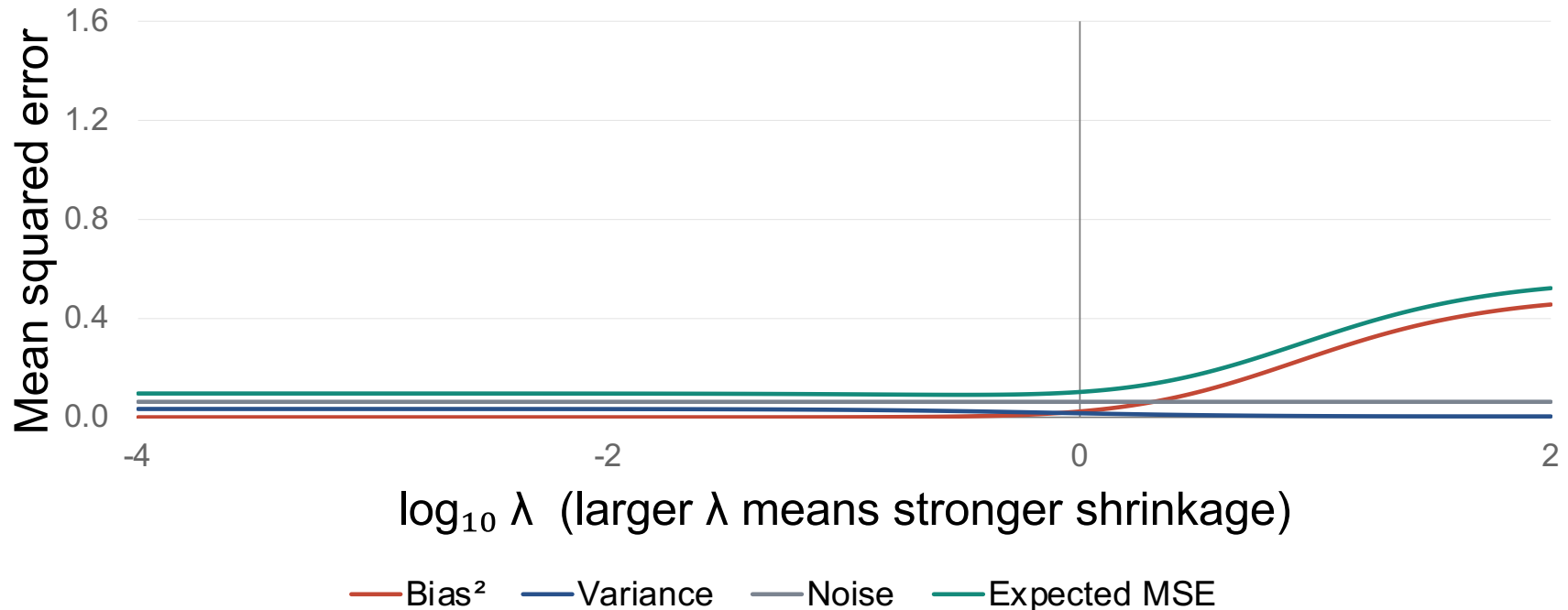
Φ contains degree-9 polynomial features. $\mathbf{w}$ is the coefficient vector.

$\lambda \geq 0$ sets ridge strength. The intercept w_0 is unpenalised.

Expected error averages over repeated data and fresh noise at the same 121 evaluation inputs.

Ridge strength and expected error

Noise standard deviation $\sigma = 0.25$. All other conditions stay fixed.

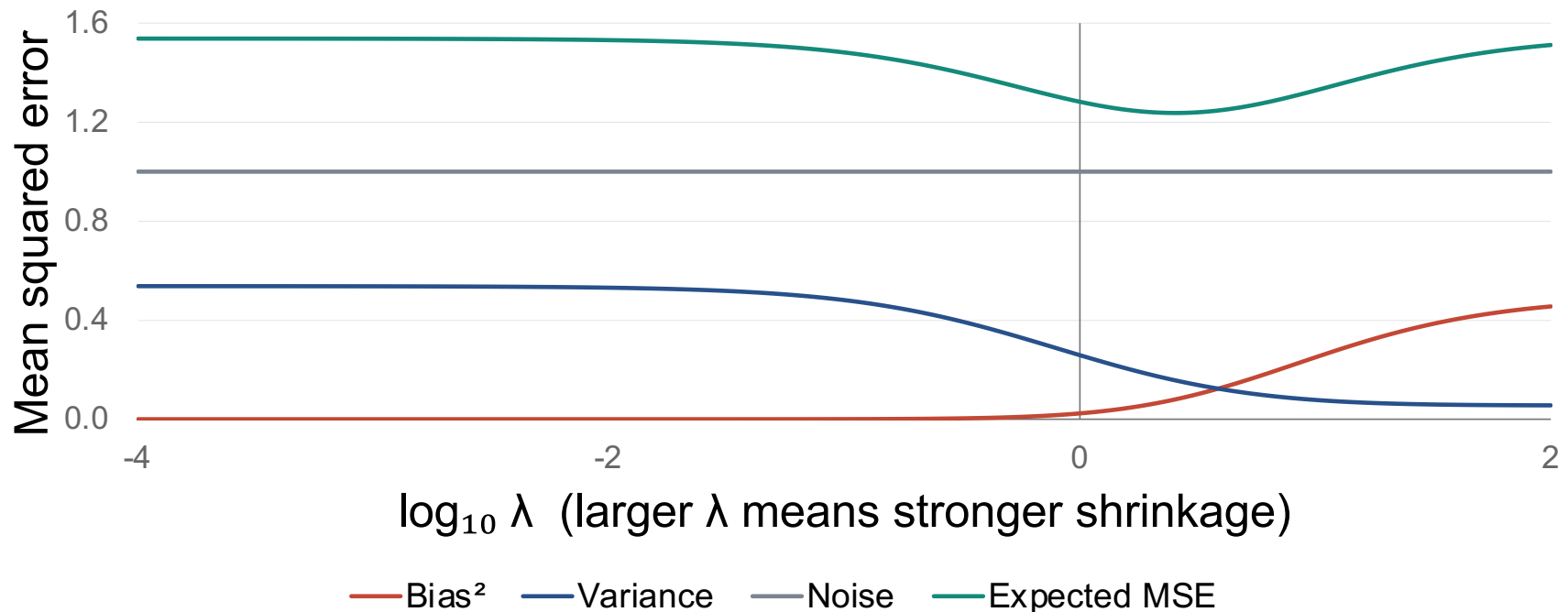


Minimum on this grid: $\lambda \approx 0.32$, expected MSE ≈ 0.091 .

Stronger shrinkage reduces variance but eventually adds too much bias.
An optimum need not lie in the interior for every problem.

More noise changes the ridge trade-off

Noise standard deviation $\sigma = 1.00$. All other conditions stay fixed.



Minimum on this grid: $\lambda \approx 2.51$, expected MSE ≈ 1.237 .

More noise increases variance and the irreducible noise floor.
The best λ rises from 0.32 to 2.51 in this controlled example.